

01. C++ basics

This chapter takes you from "I have never compiled a C++ program" to "I can write a small C++ solution with vectors, strings, maps and pairs, build it, run it, and read the compiler's complaints". Every later chapter (grids, graphs, trees) assumes exactly this and nothing more. Robots are programmed in C++ because it is fast enough to run a control loop a thousand times a second on a small on-board computer; every ROS node, motor driver and planner you will touch is written in it.

Rough time: 4 days at about 2 hours a day. Day 1 is A0 to c3 (get something compiling, learn what a box is). Day 2 is c4 to c7 plus the demo file. Days 3 and 4 are the ten problems in Part B.

What you need before this chapter: Python. You should be comfortable with lists, dicts, sets, functions and for loops in Python. Nothing else. You also need a terminal (Linux, Mac, or WSL on Windows) where typing `g++ --version` prints a version number.

Part A — The language, from zero

A0. What C++ is and why it feels different

Python is a language you talk to. You type a line, the Python interpreter reads it and does it straight away. If line 40 has a typo, you find out when the program reaches line 40.

C++ is a language you write a letter in. You write the whole program first. Then you hand the file to a translator, the *compiler* (ours is called `g++`). The compiler reads every line, checks it, and produces a brand new file in the computer's own language: an *executable*. Then you run that executable. Two steps, every time: compile, then run.

hello.cpp	----- g++ ----->	hello	----- ./hello ----->	Hello, robot
(text you typed)	(the compiler: reads, checks, translates)	(an executable: a real program the machine runs)		(what you see)

If line 40 has a typo, the compiler refuses to produce the executable and tells you the line number. Nothing runs until the whole file is right. That is annoying for the first week and a relief for the rest of your life.

C++ is typed. In Python you write `x = 5` and later `x = "five"` and nobody minds. In C++ every variable is a *box with a fixed shape*, and you say the shape when you make the box:

```
int x = 5;
```

reads as "make a box shaped for a whole number (`int`), call it `x`, put 5 in it". You cannot put "five" in that box later; the compiler stops you. This feels like paperwork at first. It is also why the compiler can catch so many mistakes before anything runs, and why the finished program is fast: the computer never has to stop and ask "what is this thing?".

The smallest useful program. Five lines that do something:

```
#include <iostream>

int main() {
    std::cout << "Hello, robot\n";
    return 0;
}
```

Line by line:

- `#include <iostream>` : "bring in the toolbox for input and output". It is Python's `import`. `iostream` is what gives you `std::cout`.
- `int main() {` : every C++ program starts running at a function called `main`. The `int` says it hands back a whole number when it finishes. The `{` opens the body.
- `std::cout << "Hello, robot\n";` : `std::cout` is the screen. `<<` means "send this to". `"\n"` is a newline. The `;` ends the statement. Python ends a statement at the end of the line; C++ ends it at the semicolon, so you can split one statement across lines.
- `return 0;` : "finished, no error". Returning 0 from `main` means success.
- `}` closes the body.

Three habits from day one.

1. **std:: in front of library things.** Everything from the standard library lives in a *namespace* called `std`, and you name it in full: `std::cout`, `std::vector`, `std::string`. Python says `math.sqrt`; C++ says `std::sqrt`. Some tutorials add `using namespace std;` at the top to skip this. Do not; it causes name clashes later and every codebase you will work in forbids it.
2. **Braces, not indentation.** Python groups lines by indenting them. C++ groups them with `{ }` and ignores indentation completely. Indent anyway, for the humans reading.
3. **Semicolons.** Every statement ends with `;`. Forgetting one is the first error you will ever get from a compiler, and the next section shows you what it looks like.

In your own words: C++ is compiled: I write the whole file, `g++` checks and translates it into an executable, and then I run that. Every variable is a box with a fixed type that I declare up front. Library names start with `std::`, blocks use braces, statements end with a semicolon.

c8. Compile and run locally with g++ (not only in the browser)

You need to be able to run things before you can learn anything else, so this comes first.

Type the hello program above into a file called `hello.cpp`. In a terminal, in the same folder, run these two lines:

```
g++ -std=c++17 -Wall -Wextra -O2 hello.cpp -o hello
./hello
```

The first line compiles. If it succeeds it prints nothing and creates a file called `hello`. The second line runs that file and prints `Hello, robot`. If the compile fails, no `hello` file is made and `g++` prints why.

What each flag means, one line each:

Flag	Meaning
<code>-std=c++17</code>	Use the 2017 version of the language. Needed for <code>auto</code> [a, b] and other things in this chapter.
<code>-Wall</code>	Turn on the common warnings (unused variable, missing return, signed/unsigned mix).
<code>-Wextra</code>	Turn on a few more. Treat every warning as a bug.
<code>-O2</code>	Optimise. Makes the program fast; without it, timing means nothing.
<code>hello.cpp</code>	The file to compile.
<code>-o hello</code>	Name the output file <code>hello</code> . Without this you get a file called <code>a.out</code> .

For the files in this repo the same command is used, with a throwaway output name:

```
cd "code/01-cpp-basics"
g++ -std=c++17 -Wall -Wextra -O2 01_two_sum.cpp -o /tmp/x && /tmp/x
```

&& means "and if that succeeded, then". So the program only runs if it compiled. Each file prints OK <filename> when all its tests pass, or stops with an assertion failure telling you which line was wrong.

Reading an error message. Break the hello program on purpose: add a variable and forget the semicolon after it.

```
#include <iostream>

int main() {
    int count = 3
    std::cout << count << "\n";
    return 0;
}
```

Save it as `bad.cpp` and compile. `g++` prints:

```
bad.cpp: In function 'int main()':
bad.cpp:5:5: error: expected ',' or ';' before 'std'
   5 |     std::cout << count << "\n";
     |     ^~~
```

How to read it:

- `bad.cpp:5:5` is *file : line : column*. The compiler is pointing at line 5, column 5.
- `error:` is the kind. An `error` means no executable was produced. A `warning` means it was produced but something looks wrong.
- `expected ';' before 'std'` is the message. It found `std` where it expected a `;`.
- The next two lines show you the source line and a `^` under the exact spot.

Notice the compiler points at line 5, but the mistake is at the *end of line 4*: that is where the semicolon belongs. The compiler only knows something is wrong when it reaches the next thing. Rule: look at the line it names, then look at the line above. The fix is `int count = 3;`.

The three messages you will see most often in this chapter:

Message	What it means	Fix
error: 'foo' was not declared in this scope	You used a name the compiler has never seen.	A typo, a missing <code>#include</code> , or you used a variable before making it.
error: 'class std::vector<int>' has no member named 'append'	You called a method that this type does not have.	Look up the C++ name (a vector's is <code>push_back</code>).
warning: comparison of integer expressions of different signedness	You compared an <code>int</code> with something like <code>v.size()</code> .	See the skip paragraph in c1; the habit is <code>static_cast<int>(v.size())</code> .

Two more rules. When there are many errors, fix the *first* one and recompile; later errors are often caused by the first. And a warning is a bug you have not met yet: fix them all.

Feeding input. The files in this repo test themselves with `assert`, so they read no input. When you want a program that reads numbers, this is the pattern:

```
#include <iostream>
#include <vector>

int main() {
    int n;
    std::cin >> n;                // read one number
    std::vector<int> v(n);        // n boxes
    for (int& x : v) std::cin >> x; // fill each box from the input
    int total = 0;
    for (int x : v) total += x;
    std::cout << "sum = " << total << "\n";
}
```

```
    return 0;
}
```

- `std::cin >> n`; reads the next whitespace-separated token from the input into `n`. It is Python's `n = int(input())`, but it does not care about line breaks.
- `std::vector<int> v(n)`; makes a list of `n` zeros (section c4).
- `for (int& x : v)` walks through the list. The `&` matters: it makes `x` be each box, so `std::cin >> x` fills the real box. Without the `&`, `x` would be a copy and `v` would stay all zeros (section c2 explains `&`).

Save as `input.cpp`, compile as before, and give it input either by typing it, by piping it, or from a file:

```
echo "3 10 20 30" | ./input
./input < input.txt
```

Both print `sum = 60`.

A minimal VS Code setup. Put this in `.vscode/tasks.json` in the project folder. Then `Ctrl+Shift+B` compiles and runs whichever `.cpp` file is open, and errors become clickable in the Problems panel.

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build & run current file",
      "type": "shell",
      "command": "g++ -std=c++17 -Wall -Wextra -O2 \"${file}\" -o /tmp/x && /tmp/x",
      "group": { "kind": "build", "isDefault": true },
      "presentation": { "reveal": "always", "clear": true },
      "problemMatcher": ["$gcc"]
    }
  ]
}
```

The Python to C++ cheat table. You will meet each of these properly in c1 to c7. This table is here so you have one place to look things up later. How to read it: the left column is the thing you would reach for in Python; the middle is what you type in C++ instead; the right column is the `#include` line you must add at the top of the file to use it. Do not memorise it now. Come back to it every time you think "how do I say X in C++".

Python	C++ (C++17)	Header
<code>list</code>	<code>std::vector<T></code>	<code><vector></code>
<code>dict</code>	<code>std::unordered_map<K, V></code> (<code>std::map</code> if you need sorted keys)	<code><unordered_map></code> / <code><map></code>
<code>set</code>	<code>std::unordered_set<T></code> (<code>std::set</code> if sorted)	<code><unordered_set></code> / <code><set></code>
<code>tuple</code>	<code>std::pair<A, B></code> / <code>std::tuple<...></code>	<code><utility></code> / <code><tuple></code>
<code>str</code>	<code>std::string</code>	<code><string></code>
<code>collections.deque</code>	<code>std::deque<T></code> ; as a queue use <code>std::queue<T></code>	<code><deque></code> / <code><queue></code>
<code>heapq (min-heap)</code>	<code>std::priority_queue<T, std::vector<T>, std::greater<T>></code> (max-heap by default)	<code><queue></code>
<code>list</code> used as a stack	<code>std::stack<T></code> or <code>std::vector<T></code> with <code>back()</code> / <code>pop_back()</code>	<code><stack></code>
<code>None</code>	<code>std::optional<T></code> , or a sentinel like <code>-1</code> / <code>nullptr</code>	<code><optional></code>
<code>len(x)</code>	<code>x.size()</code> (returns an unsigned number, see c1)	(none)
<code>x in container</code>	<code>count(x)</code> for sets and maps; <code>std::find</code> for vectors	<code><algorithm></code>

Python	C++ (C++17)	Header
<code>max(a, b), min(a, b)</code>	<code>std::max(a, b), std::min(a, b)</code> (both arguments must have the same type)	<code><algorithm></code>
<code>max(xs)</code>	<code>*std::max_element(xs.begin(), xs.end())</code>	<code><algorithm></code>
<code>sum(xs)</code>	<code>std::accumulate(xs.begin(), xs.end(), 0LL)</code> (0LL so it sums in 64 bits)	<code><numeric></code>
<code>sorted(xs) / xs.sort()</code>	<code>std::sort(xs.begin(), xs.end())</code>	<code><algorithm></code>
<code>float('inf')</code>	<code>INT_MAX / std::numeric_limits<int>::max() / 1e18</code> for long long	<code><limits> / <limits></code>
<code>abs(x)</code>	<code>std::abs(x)</code>	<code><cstdlib> / <cmath></code>
<code>x // y, x % y</code>	<code>x / y, x % y</code> (rounds toward zero, see c1)	(none)
<code>lambda x: x*2</code>	<code>[](int x) { return x * 2; }; [&]</code> to see local variables	(none)
<code>print(x)</code>	<code>std::cout << x << '\n';</code>	<code><iostream></code>
<code>assert cond</code>	<code>assert(cond);</code>	<code><cassert></code>
<code>def f(xs: list) -> int:</code>	<code>int f(const std::vector<int>& xs)</code>	(none)

(You can skip this on a first read.) While hunting a bug, compile with `g++ -std=c++17 -Wall -Wextra -g -fsanitize=address,undefined file.cpp -o /tmp/x`. `-g` adds line numbers to crash reports; `-fsanitize=address` makes reading past the end of a vector stop the program with a message instead of silently reading garbage; `-fsanitize=undefined` does the same for integer overflow. Drop them (use `-O2`) when you want to measure speed.

In your own words: I compile with `g++ -std=c++17 -Wall -Wextra -O2 file.cpp -o name` and run with `./name`. An error message is `file:line:column` and a sentence; I look at that line and the one above, fix the first error, and recompile. Warnings are bugs.

c1. Variables, types, int/long/double, casting

A type is the shape of a box. In Python a variable is a name tag tied to some object, and you can move the tag to anything. In C++ a variable *is* a box. The box has a fixed shape (its type), chosen when you make it, and only things of that shape fit.

Python:

```
count -----> 5
(a tag tied to
a thing; can be
retied to "five")
```

C++:

```
+-----+
|    5    | count
+-----+
a box shaped for whole numbers (int).
It stays that shape for its whole life.
```

The shapes you need, with their ranges in plain words:

Type	What fits in it	How big	Use it for
<code>int</code>	whole numbers	about minus 2 billion to plus 2 billion (exactly 2,147,483,647 at the top)	counts, indices, most answers
<code>long long</code>	whole numbers, bigger box	about plus or minus 9 quintillion (9 followed by 18 zeros)	sums and products that could pass 2 billion
<code>double</code>	numbers with a decimal point	enormous range, about 15 correct digits	distances, angles, anything Python calls <code>float</code>

Type	What fits in it	How big	Use it for
char	exactly one character	'a', '7', ' '; secretly a small whole number (see c5)	letters inside a string
bool	true or false	just those two	flags, yes/no answers

Making a box looks like Python with the type written in front:

```
int count = 0;           // Python: count = 0
long long total = 0;     // Python: total = 0    (Python ints just grow; C++ ones do not)
double speed = 1.5;     // Python: speed = 1.5
char grade = 'A';       // single quotes for one character; double quotes are for strings
bool done = false;      // lowercase true / false, not True / False
```

Surprise 1: dividing whole numbers gives a whole number. In Python `7 / 2` is 3.5. In C++, if both sides of `/` are `int`, the result is an `int`, and the decimals are thrown away. Python's `//` rounds *down*; C++ rounds *toward zero*. The two agree for positive numbers and disagree for negatives.

```
assert(7 / 2 == 3);       // Python: 7 / 2 == 3.5, 7 // 2 == 3
assert(-7 / 2 == -3);     // Python: -7 // 2 == -4 (down); C++ goes toward zero
assert(7 % 2 == 1);       // remainder, same as Python for positives
assert(-7 % 2 == -1);     // Python: -7 % 2 == 1; C++ keeps the sign of the left number
assert(7.0 / 2 == 3.5);  // one side is a double, so the division keeps decimals
```

The fix when you want 3.5: make one side a `double`, either by writing `7.0` or by casting (below).

Surprise 2: the box can overflow. An `int` holds up to 2,147,483,647. Think of the odometer in a car with only so many digits: add one more mile and it rolls over.

```
2,147,483,647 + 1 -> rolls over to -2,147,483,648    (in an int box)
100,000 * 100,000 = 10,000,000,000 which does not fit in an int at all
```

Python never does this because its integers grow as needed. In C++ the rule is: if a number *might* pass 2 billion, use `long long`, and widen *before* the arithmetic, not after:

```
int a = 100000, b = 100000;
long long product = static_cast<long long>(a) * b; // a is widened first, so the multiply is done in the big box
assert(product == 10000000000LL);
```

Writing `long long bad = a * b;` instead does the multiply in `int` (overflow happens there) and only then copies the wrong number into the big box. The `LL` on `10000000000LL` tells the compiler the number itself is a `long long`.

Casting: telling the compiler to change the box shape. `static_cast<T>(x)` reads as "make me a `T` out of `x`". You just saw it widen an `int` to `long long`. Two more uses:

```
double d = 3.99;
int whole = static_cast<int>(d); // 3: cuts off the decimals, it does not round
assert(whole == 3);
double half = static_cast<double>(7) / 2; // 3.5: the 7 became a double before dividing
assert(half == 3.5);
```

C++ also converts on its own in some places, silently: `int x = 3.99;` compiles and stores 3, `bool b = 42;` stores `true`, and `true + true` is 2. `-Wall -Wextra` do not warn about most of these, so read your types.

(You can skip this on a first read.) `v.size()` on a vector does not return an `int`. It returns `size_t`, an *unsigned* type that cannot be negative. Two things follow. Comparing `int i` with `v.size()` gives the "different signedness" warning from `c8`; the habit is to convert once: `int n = static_cast<int>(v.size());` and compare with `n`. And `v.size() - 1` on an empty vector is not `-1`; the unsigned box rolls over the other

way to 18,446,744,073,709,551,615. Cast to `int` before subtracting: `static_cast<int>(v.size()) - 1`. Also, overflow of a signed `int` is officially "undefined behaviour": the compiler is allowed to assume it never happens, so the rolled-over odometer above is what usually happens, not what is guaranteed.

Try it: in `00_basics_demo.cpp`, change `assert(-7 / 2 == -3);` to `assert(-7 / 2 == -4);`. Predict what the program prints before you compile and run it.

In your own words: A type is the fixed shape of a variable's box. `int` holds up to about 2 billion, `long long` up to about 9 quintillion, `double` holds decimals. Dividing two ints throws away the decimals and rounds toward zero. If a result might pass 2 billion I use `long long` and widen with `static_cast` before the arithmetic.

c2. Functions, pass by value vs reference (`int&`)

The shape of a function. Python:

```
def add(a, b):
    return a + b
```

C++:

```
int add(int a, int b) {
    return a + b;
}
```

Read it as: "a function that hands back an `int`, called `add`, taking two `int` boxes `a` and `b`". The return type comes first. Every parameter has a type. A function that hands back nothing says `void` where the `int` is. Functions must be written *above* the place they are used (or declared above; for now, just put helpers above `main`).

Two boxes. When you call `add(2, 3)`, what exactly does the function get? In C++ there are two answers, and you choose with one character.

By value: `void f(int x)`

```
caller's box      f's box
+-----+ copy  +-----+
|  5  | -----> |  5  |
+-----+      +-----+
   n              x
```

f does `x += 1`: `n` is still 5

By reference: `void f(int& x)`

```
caller's box
+-----+
|  5  | <--- x is a second name
+-----+    tied to this same box
   n      x
```

f does `x += 1`: `n` is now 6

By value (`int x`): the function gets a **copy** in a brand new box. Whatever it does to `x`, the caller's `n` never changes. This is the default.

By reference (`int& x`): the `&` means "do not copy; let `x` be another name for the caller's box". Changes to `x` are changes to `n`.

```
void addOneCopy(int x) { x += 1; }    // works on a private copy
void addOneRef(int& x) { x += 1; }    // works on the caller's own box
```

```
int n = 5;
addOneCopy(n);
assert(n == 5);    // the copy was changed, n was not
addOneRef(n);
assert(n == 6);    // x was another name for n
```

const& means "look but do not touch". Copying an `int` is nothing. Copying a vector of a million numbers is a million copies, every call. So for anything bigger than a number, you pass by reference to avoid the copy, and you add `const` to promise you will not change it. The compiler enforces the promise: try to modify a `const&` parameter and you get an error.

```
int sum(const std::vector<int>& v) {    // no copy is made; v cannot be changed inside
    int s = 0;
    for (int x : v) s += x;
    return s;
}
```

The three forms, side by side:

You write	Copied?	Function can change the caller's data?	Use for
<code>int x / char c / double d</code>	yes, but it is tiny	no	numbers, characters, bools
<code>std::vector<int>& v</code>	no	yes	when changing the caller's data is the point (sort it in place, fill it in)
<code>const std::vector<int>& v</code>	no	no	anything big that you only read: vectors, strings, maps. This is the default for big things

The Python intuition, and where it breaks. In Python, a function that gets a list can change the caller's list, because both names point at the same object:

```
def f(xs):
    xs.append(99)

a = [1, 2, 3]
f(a)
print(len(a))    # 4: the caller's list changed
```

In C++ that is only true if you write the `&`:

```
void byValue(std::vector<int> v) { v.push_back(99); }    // v is a full copy; the caller sees nothing
void byRef(std::vector<int>& v) { v.push_back(99); }    // v is the caller's vector
```

```
std::vector<int> a = {1, 2, 3};
byValue(a);
assert(a.size() == 3);    // unchanged, unlike Python
byRef(a);
assert(a.size() == 4);    // changed, like Python
```

It breaks the other way too. In Python, a function can never change the caller's *integer*: `def g(x): x += 1` leaves the caller's `x` alone. In C++, `void g(int& x) { x += 1; }` does change it. So in C++ you never guess from the type of the thing; you read the parameter declaration. `&` means shared, no `&` means copy.

(You can skip this on a first read.) Returning a vector *from* a function by value (`std::vector<int> makeSquares(int n)`) is fine and does not copy in practice: the compiler hands the caller the same buffer. What you must never do is return a reference to a local variable; the local is destroyed when the function ends and the reference points at nothing.

Try it: in `00_basics_demo.cpp`, delete the `&` in `void byRef(std::vector<int>& v)`. Predict which `assert` fails before you compile and run.

In your own words: A parameter without & is a copy in a new box; the caller never sees changes. A parameter with & is a second name for the caller's box. For anything bigger than a number I write `const T&` so there is no copy and I cannot change it, and plain `T&` only when changing the caller's data is the whole point.

c3. if/for/while, range-for, auto

Control flow is the Python you know with braces and parentheses. Side by side:

```
if x > 0 and x < 10:
    print("small")
elif x == 0:
    print("zero")
else:
    print("other")
```

```
if (x > 0 && x < 10) {
    std::cout << "small\n";
} else if (x == 0) {
    std::cout << "zero\n";
} else {
    std::cout << "other\n";
}
```

The differences: the condition must be in `( )`; and is `&&`, or is `||`, not is `!`; the body is in `{ }`; and there is no chained comparison, so Python's `0 < x < 10` must be written `x > 0 && x < 10`. `while` is the same story:

```
int lo = 0, hi = 10;
while (lo < hi) {
    ++lo;
}
```

The three kinds of for loop.

1. **Counting.** Python `for i in range(n):` is

```
cpp for (int i = 0; i < n; ++i) { // use i }
```

Three parts separated by `;`: *start* (`int i = 0`), *keep going while* (`i < n`), *step after each round* (`++i`, which is `i += 1`). To count down: `for (int i = n - 1; i >= 0; --i)`.

1. **Range-for by copy.** Python `for x in v:` is

```
cpp for (int x : v) { // x is a copy of each element in turn }
```

Read `:` as "in". Because `x` is a copy, changing `x` does not change `v`.

1. **Range-for by reference.** Add `&` and `x` becomes each element itself:

```
cpp for (int& x : v) { x *= 2; // this changes v }
```

`break` and `continue` work exactly as in Python.

auto means "you work out the type". Instead of writing the type of a box, you can write `auto` and the compiler looks at the right-hand side and picks the shape:

```
auto n = v.size();           // n gets whatever type size() returns
auto it = seen.find(5);      // it gets the iterator type, which is long to spell out
```

Use it when the type is long or boring to type. Do not use it when the type carries meaning you would otherwise state (`int count = 0;` is clearer than `auto count = 0;`).

auto versus auto&: copy versus alias. This is the trap that bites everyone once. In a range-for, auto gives you a *copy* of each element, auto& gives you the element itself, exactly like `int` versus `int&` above. With a grid (a vector of rows) it matters a lot:

grid, the real rows <pre> +---+---+---+ 0 0 0 row 0 +---+---+---+ 0 0 0 row 1 +---+---+---+ </pre>	for (auto row : grid) row is a COPY of the row: <pre> +---+---+---+ 0 0 0 (new boxes) +---+---+---+ </pre> row[0] = 1 changes the copy, then the copy is thrown away.	for (auto& row : grid) row is another name for the real row. No new boxes. row[0] = 1 changes grid.
---	--	--

```

std::vector<std::vector<int>> grid(2, std::vector<int>(3, 0));
for (auto row : grid) row[0] = 1;      // each row is copied; the copies are changed and thrown away
assert(grid[0][0] == 0);
for (auto& row : grid) row[0] = 1;      // row IS the grid's row
assert(grid[0][0] == 1);

```

The rule: `const auto&` when you only read, `auto&` when you change, plain `auto` only for small things like `int` and `char`. Written out: `for (const auto& row : grid)` to read rows.

The one-line if. Python's `-1 if x < 0 else 1` is `(x < 0) ? -1 : 1`. Read `?` as "then" and `:` as "else".

(You can skip this on a first read.) There is no `for ... else` in C++; use a `bool found` flag or put the loop in a function and `return` from inside it. There is also no way to `break` out of two nested loops at once; again, put them in a function and `return`.

In your own words: `if`, `while` and the counting `for` are Python with parentheses and braces. `for (int x : v)` walks a container by copy, `for (int& x : v)` by reference. `auto` asks the compiler to pick the type, and `auto&` in a loop means "the real element, not a copy".

c4. `std::vector` — `push_back`, `size`, indexing, 2D vectors

A `std::vector` is a Python list that holds one type. It is a row of boxes, all the same shape, numbered from 0, that can grow at the end.

```

std::vector<int> v = {4, 8, 15};

index:      0      1      2
v :         +-----+-----+-----+
           |  4  |  8  | 15  |         v.size() == 3
           +-----+-----+-----+

v.push_back(16);

v :         +-----+-----+-----+-----+
           |  4  |  8  | 15  | 16  |         v.size() == 4
           +-----+-----+-----+-----+

```

`std::vector<int>` reads as "vector of int". The thing in the angle brackets is the shape of every box. `std::vector<std::string>` holds strings, `std::vector<bool>` holds flags. You need `#include <vector>` at the top of the file.

Every common operation, side by side with Python:

Python	C++	Note
<code>v = []</code>	<code>std::vector<int> v;</code>	empty
<code>v = [4, 8, 15]</code>	<code>std::vector<int> v = {4, 8, 15};</code>	

Python	C++	Note
<code>v = [0] * n</code>	<code>std::vector<int> v(n, 0);</code>	n boxes, each holding 0. Round brackets.
<code>v.append(x)</code>	<code>v.push_back(x);</code>	
<code>v.pop()</code>	<code>int x = v.back(); v.pop_back();</code>	two steps: look at the last one, then remove it. <code>pop_back()</code> returns nothing
<code>v[-1]</code>	<code>v.back()</code>	no negative indexing in C++
<code>v[0]</code>	<code>v.front()</code> or <code>v[0]</code>	
<code>len(v)</code>	<code>v.size()</code>	unsigned, see c1
<code>not v</code>	<code>v.empty()</code>	
<code>v[i]</code>	<code>v[i]</code>	no bounds check: <code>v[100]</code> on a 3-element vector does not raise, it reads garbage or crashes
<code>v[i] (checked)</code>	<code>v.at(i)</code>	throws an exception if out of range; use while debugging
<code>v.sort()</code>	<code>std::sort(v.begin(), v.end());</code>	needs <code>#include <algorithm></code>
<code>v[::-1] (in place)</code>	<code>std::reverse(v.begin(), v.end());</code>	<code><algorithm></code>
<code>x in v</code>	<code>std::find(v.begin(), v.end(), x) != v.end()</code>	<code><algorithm></code> ; a linear scan, like Python
<code>v[1:3]</code>	<code>std::vector<int>(v.begin() + 1, v.begin() + 3)</code>	a copy of positions 1 and 2
<code>v.clear()</code>	<code>v.clear();</code>	

`v.begin()` and `v.end()` are "the start" and "one past the last"; the algorithm functions take that pair to mean "the whole vector". Just copy the pattern.

```
std::vector<int> v;
v.push_back(4);
v.push_back(8);
v.push_back(15);
assert(v.size() == 3);
assert(v[0] == 4 && v.back() == 15);
int last = v.back();
v.pop_back();
assert(last == 15 && v.size() == 2);
std::vector<int> zeros(5, 0);           // five boxes of 0
assert(zeros.size() == 5 && zeros[4] == 0);
std::sort(v.begin(), v.end());
assert(v == std::vector<int>({4, 8})); // you can compare whole vectors with ==
```

The bracket trap. `std::vector<int> a(5, 0)` with round brackets means "5 boxes of 0". `std::vector<int> b{5, 0}` with curly brackets means "two boxes, holding 5 and 0". Use round brackets for (count, value) and curly for a literal list.

A 2D vector is a vector of rows. A grid with 3 rows and 4 columns is a vector holding 3 vectors, each holding 4 ints:

```
std::vector<std::vector<int>> grid(3, std::vector<int>(4, 0));
```

	c=0	c=1	c=2	c=3	
r=0	0	0	0	0	<- grid[0], a vector<int> of size 4
r=1	0	0	0	0	<- grid[1]

```

r=2      +-----+-----+-----+-----+
        | 0  | 0  | 0  | 0  |  <- grid[2]
        +-----+-----+-----+-----+

grid[1][2] = 7;   row 1, then column 2

```

That construction line, word by word:

- `std::vector<std::vector<int>>` : the type. "A vector whose boxes are each a vector of int."
- `grid` : the name.
- `(3, ...)` : make 3 boxes (3 rows), and fill each one with a copy of ...
- `std::vector<int>(4, 0)` : ... a vector of 4 ints, each 0. That is one row.

So it is Python's `grid = [[0] * 4 for _ in range(3)]`. Getting the size back:

```

std::vector<std::vector<int>> grid(3, std::vector<int>(4, 0));
grid[1][2] = 7;
int rows = static_cast<int>(grid.size());           // 3
int cols = static_cast<int>(grid[0].size());         // 4, the length of the first row
assert(rows == 3 && cols == 4 && grid[1][2] == 7);

```

`grid[r][c]` means row `r`, then column `c`, exactly as Python's list of lists. Row first.

(You can skip this on a first read.) `v.reserve(1000)` tells the vector to get room for 1000 elements now, so the next 1000 `push_backs` do not have to grow the storage; `v.size()` is still 0 after it. Each row of a 2D vector is its own separate block of memory, which is fine for everything in this tutorial.

Try it: in `00_basics_demo.cpp`, change `v.at(100)` to `v[100]` inside the `try` block. Predict what happens to `assert(throw)`. Then run it: either the program crashes, or `v[100]` quietly reads garbage, nothing is thrown, and the `assert` fails. `[]` does not check.

In your own words: `std::vector<T>` is a list where every element has type `T`. `push_back` appends, `size()` is the length, `[]` indexes from 0 with no bounds check. `std::vector<int> v(n, 0)` is `n` zeros, and `std::vector<std::vector<int>> g(rows, std::vector<int>(cols, 0))` is a grid of zeros indexed `g[r][c]`.

c5. `std::string` — `substr`, `find`, `compare`, `to_string`/`stoi`

A `std::string` is a vector of characters with some extras. Unlike a Python string, you can change it in place: `s[0] = 'H'` works, `s += "!"` appends. You need `#include <string>`.

```

std::string s = "hello";

index:    0      1      2      3      4
s :       +-----+-----+-----+-----+
          | 'h' | 'e' | 'l' | 'l' | 'o' |   s.size() == 5
          +-----+-----+-----+-----+

```

Python	C++	Note
<code>s = "hello"</code>	<code>std::string s = "hello";</code>	double quotes for strings, single for one <code>char</code>
<code>len(s)</code>	<code>s.size()</code>	
<code>s[i]</code>	<code>s[i]</code>	gives a <code>char</code> ; no negative indexing
<code>s + t</code>	<code>s + t</code>	
<code>s += "x"</code>	<code>s += "x";</code> or <code>s.push_back('x');</code>	cheap, unlike Python where strings are immutable
<code>s[2:5]</code>	<code>s.substr(2, 3)</code>	(start, length) , not (start, end): 3 characters starting at 2
<code>s[6:]</code>	<code>s.substr(6)</code>	from 6 to the end

Python	C++	Note
<code>s.find("lo")</code>	<code>s.find("lo")</code>	returns the index, or <code>std::string::npos</code> if absent (Python returns -1)
<code>"lo" in s</code>	<code>s.find("lo") != std::string::npos</code>	
<code>s == t,</code> <code>s < t</code>	<code>s == t, s < t</code>	dictionary order, same as Python
<code>str(42)</code>	<code>std::to_string(42)</code>	gives "42"
<code>int("42")</code>	<code>std::stoi("42")</code>	"string to int"; <code>std::stod</code> for a double, <code>std::stoll</code> for long long
<code>s[::-1]</code>	<code>std::string r(s.rbegin(), s.rend());</code>	a reversed copy; or <code>std::reverse(s.begin(), s.end())</code> in place
<code>for ch in s:</code>	<code>for (char ch : s)</code>	
<code>s.split()</code>	no one-liner; see the skip paragraph	

```
std::string s = "hello world";
assert(s.size() == 11);
assert(s.substr(6) == "world");
assert(s.substr(0, 5) == "hello");           // position 0, length 5
assert(s.find("wor") == 6);
assert(s.find('z') == std::string::npos);    // not found
assert(std::to_string(42) == "42");
assert(std::stoi("42") + 1 == 43);
s += "!";
assert(s.back() == '!');
std::string r(s.rbegin(), s.rend());
assert(r == "!dlrow olleh");
```

The '0' trick: a char is secretly a number. Every character is stored as a number, its ASCII code. The digits and the lowercase letters are stored in order:

```
character:  '0'  '1'  '2'  '3'  '4'  '5'  '6'  '7'  '8'  '9'
its number:  48   49   50   51   52   53   54   55   56   57

character:  'a'  'b'  'c'  'd'  ...  'z'
its number:  97   98   99  100  ...  122
```

So subtracting characters subtracts their numbers, and that turns a digit character into the digit, or a letter into its position in the alphabet:

```
'7' - '0' = 55 - 48 = 7      the digit seven, as a number
'c' - 'a' = 99 - 97 = 2      c is the 3rd letter, position 2 from 0
'a' + 2   = 97 + 2  = 99    which is 'c'   (going back the other way)
```

```
char c = '7';
assert(c - '0' == 7);
assert('c' - 'a' == 2);
assert(static_cast<char>('a' + 2) == 'c');    // the addition gives an int; cast it back to char
int count[26] = {};                          // 26 boxes of 0, one per letter; the {} zeroes them
for (char ch : std::string("banana")) ++count[ch - 'a'];
assert(count['a' - 'a'] == 3 && count['n' - 'a'] == 2);
```

That last pattern, "26 boxes indexed by `ch - 'a'`", is how you count letters without a dictionary. You will use it in Valid Anagram below. Python's `ord(ch) - ord('a')` is the same idea with the numbers made visible.

The find trap. `s.find('z')` returns `std::string::npos` when the character is absent. `npos` is a huge unsigned number, not `-1`, so `if (s.find('z') >= 0)` is always true. Always compare with `npos`.

(You can skip this on a first read.) Splitting on whitespace uses a string stream: `std::istringstream in(s); std::string word; while (in >> word) words.push_back(word);` with `#include <sstream>`. To split on a specific character use `while (std::getline(in, word, ','))`. `std::isdigit(c)`, `std::isalpha(c)` and `std::tolower(c)` live in `<cctype>`.

In your own words: `std::string` is a changeable vector of `char`. `substr` takes a start and a *length*. `find` returns `npos` when absent, so I compare with `npos`, not `-1`. A `char` is a small number, so `ch - '0'` is the digit and `ch - 'a'` is the letter's position.

c6. `std::unordered_map` and `unordered_set`

An `unordered_map` is a Python dict. An `unordered_set` is a Python set. Both are hash tables: looking something up takes the same short time no matter how many things are inside. Both need to know the type of what they hold. Headers: `<unordered_map>` and `<unordered_set>`.

```
std::unordered_map<std::string, int> ages;      key: string,  value: int
```

```

      key      ->  value
+-----+ +-----+
| "ann" | -> | 30 |
+-----+ +-----+
| "bob" | -> | 25 |
+-----+ +-----+
```

Python	C++	Note
<code>d = {}</code>	<code>std::unordered_map<std::string, int> d;</code>	
<code>d[k] = v</code>	<code>d[k] = v;</code>	
<code>d[k]</code> (read)	<code>d[k]</code>	inserts k with value 0 if absent , see the trap
<code>k in d</code>	<code>d.count(k)</code>	gives 0 or 1, never inserts
<code>d.get(k)</code>	<code>auto it = d.find(k); if (it != d.end()) use(it->second);</code>	<code>find</code> gives an iterator; <code>d.end()</code> means "not there"
<code>d[k] += 1</code> on a <code>defaultdict(int)</code>	<code>d[k] += 1;</code>	the auto-insert is exactly what you want here
<code>del d[k]</code>	<code>d.erase(k);</code>	
<code>for k, v in d.items():</code>	<code>for (const auto& [k, v] : d)</code>	see below
<code>len(d)</code>	<code>d.size()</code>	
<code>s = set()</code>	<code>std::unordered_set<int> s;</code>	
<code>s.add(x)</code>	<code>s.insert(x);</code>	
<code>x in s</code>	<code>s.count(x)</code>	0 or 1
<code>s.remove(x)</code>	<code>s.erase(x);</code>	

Order of iteration is arbitrary in both languages (Python dicts keep insertion order since 3.7; C++ unordered containers do not).

The [] trap, drawn. In Python, reading `d["bob"]` when "bob" is missing raises `KeyError`. In C++, `d["bob"]` when "bob" is missing **creates** the entry with value 0 and hands it back. Reading looks harmless and silently changes the map:

```

before:    ages = { "ann" -> 30 }                ages.size() == 1

           if (ages["bob"] > 18) { ... }    <- just a look, surely?

after:     ages = { "ann" -> 30,  "bob" -> 0 }      ages.size() == 2
           ^^^^^^^^^^
           invented by the [] lookup

```

```

std::unordered_map<std::string, int> ages;
ages["ann"] = 30;
assert(ages.size() == 1);
assert(ages.count("bob") == 0);           // count never inserts
int bobsAge = ages["bob"];                // this DOES insert bob -> 0
assert(bobsAge == 0 && ages.size() == 2);

```

So: use `[]` when you want to write or you want the auto-insert (counting). Use `count` or `find` when you only want to know whether something is there. In Two Sum below, using `[]` for the lookup would make you "find" a number you never stored.

Iterating with `auto& [k, v]`. Each entry of a map is a pair (c7) of key and value. The cleanest loop names both:

```

std::unordered_map<std::string, int> freq;
for (char ch : std::string("banana")) freq[std::string(1, ch)] += 1;
int total = 0;
for (const auto& [letter, n] : freq) { // letter is the key, n is the value
    assert(letter.size() == 1);
    total += n;
}
assert(total == 6 && freq["a"] == 3 && freq["n"] == 2 && freq["b"] == 1);

```

`const auto&` so each entry is looked at, not copied. Without the `&`, every pair is copied on every round of the loop.

`insert` tells you whether it was new. For a set, `s.insert(x)` returns a pair; its `.second` is `true` if `x` was actually added and `false` if it was already there. That saves a separate `count` call:

```

std::unordered_set<int> seen;
assert(seen.insert(5).second == true);    // 5 was new
assert(seen.insert(5).second == false);   // 5 was already there
assert(seen.size() == 1);

```

(You can skip this on a first read.) `std::pair<int,int>` cannot be a key of an unordered container out of the box (there is no built-in hash for it; the compile error is a page of template text mentioning `std::hash`). For grid coordinates use a `std::vector<std::vector<bool>>` `visited` instead, or encode `r * cols + c` into one `int`. `std::map` and `std::set` are the sorted cousins: a bit slower, but iteration comes out in key order.

In your own words: `unordered_map` is a dict and `unordered_set` is a set. `d[k]` inserts a default when `k` is missing, so to test membership I use `count` or `find`. I iterate with `for (const auto& [k, v] : d)`. `s.insert(x).second` tells me whether `x` was new.

c7. `std::pair`, `std::tuple`, structured bindings

A `std::pair` is a tuple with exactly two slots, and the slots have fixed types. Python's `(3, "cat")` becomes:

```

std::pair<int, std::string> p = {3, "cat"};

+-----+-----+
|   3   | "cat" |

```

```
+-----+-----+
  .first  .second
```

The slots are called `.first` and `.second`, not `[0]` and `[1]`. Header: `<utility>` (but `<vector>` and `<map>` already pull it in).

```
std::pair<int, std::string> p = {3, "cat"};
assert(p.first == 3 && p.second == "cat");
std::pair<int, int> q = std::make_pair(1, 2);    // make_pair works out the types for you
assert(q.first == 1 && q.second == 2);
std::vector<std::pair<int, int>> edges;
edges.push_back({0, 1});                       // {a, b} builds a pair where one is expected
edges.emplace_back(1, 2);                       // same thing, built in place
assert(edges.size() == 2 && edges[1].second == 2);
```

Comparing pairs: first slot, then second. Pairs compare like words in a dictionary: compare `.first`; only if those are equal, compare `.second`. This is what makes a vector of pairs sort sensibly with no extra work, and why a queue of (priority, item) pairs orders by priority.

```
(1, 5) < (2, 0)    because 1 < 2, the second slots are never looked at
(1, 5) < (1, 7)    first slots tie, then 5 < 7
(2, 0) == (2, 0)
```

```
assert(std::make_pair(1, 5) < std::make_pair(2, 0));
assert(std::make_pair(1, 5) < std::make_pair(1, 7));
assert(std::make_pair(2, 0) == std::make_pair(2, 0));
std::vector<std::pair<int, int>> pts = {{2, 0}, {1, 7}, {1, 5}};
std::sort(pts.begin(), pts.end());
assert(pts[0] == std::make_pair(1, 5) && pts[2] == std::make_pair(2, 0));
```

A `std::tuple` is the same with any number of slots. `std::tuple<int, int, double> t = {1, 2, 3.5};` and you read slot `i` with `std::get<i>(t)`, where `i` must be a literal number. Header `<tuple>`. You will rarely need one; a small struct with named fields is usually clearer.

Structured bindings: Python's `a, b = p`. With `auto [a, b] = p`; C++ makes two new variables `a` and `b` holding the two slots. It works on pairs, tuples and simple structs, and it is what makes map loops readable:

```
std::pair<int, int> cell = {4, 7};
auto [r, c] = cell;           // r == 4, c == 7, two new variables
assert(r == 4 && c == 7);
std::tuple<int, int, double> t = {1, 2, 3.5};
assert(std::get<2>(t) == 3.5);
auto [x, y, w] = t;
assert(x == 1 && y == 2 && w == 3.5);
std::vector<std::pair<int, int>> pairs = {{1, 2}, {3, 4}};
int sum = 0;
for (const auto& [a, b] : pairs) sum += a * b;    // Python: for a, b in pairs
assert(sum == 14);
```

Python	C++
<code>t = (1, 2)</code>	<code>std::pair<int, int> t = {1, 2};</code> or <code>std::make_pair(1, 2)</code>
<code>a, b = t</code>	<code>auto [a, b] = t;</code>
<code>for a, b in pairs:</code>	<code>for (const auto& [a, b] : pairs)</code>
<code>t[0], t[1]</code>	<code>t.first, t.second</code>
<code>sorted(pairs)</code>	<code>std::sort(pairs.begin(), pairs.end())</code> (same first-then-second order)

(You can skip this on a first read.) A structured binding always declares new variables; you cannot bind into variables that already exist (that is `std::tie(a, b) = p;`), and you cannot skip a slot (`auto [_, b]` still declares `_`). `std::make_pair(1, "x")` makes the second slot a `const char*`, not a `std::string`; when you want a string, write the type out.

In your own words: A pair is a two-slot tuple with `.first` and `.second`. Pairs compare by the first slot, then the second, so vectors of pairs sort with no extra code. `auto [a, b] = p;` unpacks a pair into two named variables, and `for (const auto& [k, v] : map)` walks a map.

A9. The demo file

Every surprise from c1 to c7 is written down as an `assert` in one file, so you can read the numbers instead of taking my word for it. Compile and run it:

```
g++ -std=c++17 -Wall -Wextra -O2 00_basics_demo.cpp -o /tmp/x && /tmp/x
```

```
// 00_basics_demo.cpp – the C++ traps that bite Python developers first.
// Every block asserts what actually happens, so you can read the numbers
// rather than take my word for it.
#include <cassert>
#include <climits>
#include <cstdint>
#include <iostream>
#include <string>
#include <vector>

// ---- pass by value vs by reference ----
void byValue(std::vector<int> v) { v.push_back(99); } // copies the vector
void byRef(std::vector<int>& v) { v.push_back(99); } // aliases the caller's
int sumConstRef(const std::vector<int>& v) { // no copy, can't modify
    int s = 0;
    for (int x : v) s += x;
    return s;
}

// Returning a vector by value is fine: the compiler moves/elides the copy.
std::vector<int> makeSquares(int n) {
    std::vector<int> out;
    out.reserve(n);
    for (int i = 0; i < n; ++i) out.push_back(i * i);
    return out;
}

int main() {
    // ---- integer division and modulo ----
    assert(7 / 2 == 3); // Python: 7 / 2 == 3.5, 7 // 2 == 3
    assert(-7 / 2 == -3); // truncates toward zero; Python: -7 // 2 == -4
    assert(-7 % 2 == -1); // sign follows the dividend; Python: -7 % 2 == 1
    assert(7.0 / 2 == 3.5); // one double operand makes it floating division
    assert(static_cast<double>(7) / 2 == 3.5);

    // ---- int overflow (int is 32-bit here; Python ints are unbounded) ----
    int big = INT_MAX; // 2147483647
    long long safe = static_cast<long long>(big) + 1; // widen BEFORE adding
    assert(safe == 2147483648LL);
    // `big + 1` as int would be signed overflow = undefined behaviour.
    // -fsanitize=undefined reports it at runtime; -O2 may "optimise" it away.
    int a = 100000, b = 100000;
    long long product = static_cast<long long>(a) * b; // 1e10 does not fit int
    assert(product == 10000000000LL);

    // ---- size_t subtraction ----
    std::vector<int> empty;
    // empty.size() - 1 is size_t and wraps to 18446744073709551615, not -1.
```

```

assert(empty.size() - 1 > 1000000);
// The idiom: cast to int first, then subtract.
int lastIdx = static_cast<int>(empty.size()) - 1;
assert(lastIdx == -1);
// Loop that would run "forever" on an empty vector: for (size_t i = 0; i <= v.size() - 1; ++i)
// Loop that is safe: for (int i = 0; i < (int)v.size(); ++i)

// ---- char arithmetic ----
char c = '7';
assert(c - '0' == 7); // char is a small integer; '7' - '0' == 7
assert('c' - 'a' == 2); // index into a 26-bucket array
assert(static_cast<char>('a' + 2) == 'c');
int count[26] = {}; // zero-initialised
for (char ch : std::string("banana")) ++count[ch - 'a'];
assert(count['a' - 'a'] == 3 && count['n' - 'a'] == 2);

// ---- implicit conversions ----
double d = 3.99;
int truncated = static_cast<int>(d); // 3, not 4 - truncation, not rounding
assert(truncated == 3);
int fromBool = true + true; // bool promotes to int
assert(fromBool == 2);
bool fromInt = 42; // any non-zero -> true
assert(fromInt);

// ---- pass by value vs by reference ----
std::vector<int> v = {1, 2, 3};
byValue(v);
assert(v.size() == 3); // caller unchanged (Python would see 4)
byRef(v);
assert(v.size() == 4); // caller sees the push
assert(sumConstRef(v) == 105);
std::vector<int> sq = makeSquares(4);
assert(sq == std::vector<int>({0, 1, 4, 9}));

// ---- auto copy vs auto& ----
std::vector<std::vector<int>> grid(2, std::vector<int>(3, 0));
for (auto row : grid) row[0] = 1; // `row` is a COPY; grid untouched
assert(grid[0][0] == 0 && grid[1][0] == 0);
for (auto& row : grid) row[0] = 1; // reference; grid modified
assert(grid[0][0] == 1 && grid[1][0] == 1);
for (const auto& row : grid) assert(row.size() == 3); // read-only, no copy

// ---- operator[] does not bounds-check ----
// v[100] compiles and is undefined behaviour. v.at(100) throws
// std::out_of_range. Use .at() while debugging, [] once correct.
bool threw = false;
try { v.at(100); } catch (const std::out_of_range&) { threw = true; }
assert(threw);

std::cout << "OK 00_basics_demo.cpp\n";
return 0;
}

```

What to look at:

- The first block is c1: integer division, `-7 / 2`, and `7.0 / 2`. Each `assert` states what C++ does and the comment states what Python does.
- The overflow block widens with `static_cast<long long>` before the `+ 1` and the `* b`. `INT_MAX` (from `<climits>`) is 2,147,483,647, the top of the `int` box.
- The `size_t` block is the c1 skip paragraph made concrete: `empty.size() - 1` is a huge number, and the fix is to cast to `int` first.
- The `char` block is the `'0'` trick from c5, including the 26-box letter count.
- `byValue`, `byRef` and `sumConstRef` at the top are the three parameter forms from c2, and the asserts in `main` show which one changed the caller's vector.

- The `auto` versus `auto&` block is the c3 drawing as code: the first loop changes copies and the grid is untouched.
- The last block shows `.at()` throwing where `[]` would silently misbehave. `try / catch` is Python's `try / except`; you do not need it anywhere else in this chapter.

Then break it. Change one number in an `assert`, predict the message, and run it. An assertion failure prints the file, the line, and the condition that was false; get used to reading that.

Part B — The problems

Each problem follows the same shape: the full problem in plain words with a drawn example, how you would solve it by hand, the Python you would have written, the C++ solution with every new thing explained, a walk-through of the example, and the follow-up questions that usually come with it.

The rule for using this part: read the problem statement, close the file, and try it in C++ for 25 minutes. Then read the rest. You already know these problems from Python; the point is to produce the C++ without stalling on syntax.

1. Two Sum (LeetCode 1, easy)

The problem in plain words. You are given a list of whole numbers `nums` and a number `target`. Find two *different positions* in the list whose numbers add up to `target`, and return those two positions. There is exactly one answer. You may not use the same position twice, but two different positions may hold the same number.

```
nums = [ 2,  7, 11, 15 ]      target = 9
index  0   1   2   3

2 + 7 = 9, at positions 0 and 1  -> answer [0, 1]

nums = [3, 2, 4], target = 6    -> 2 + 4 at positions 1 and 2  -> [1, 2]
                                   (not [0, 0]: position 0 used twice is not allowed)
nums = [3, 3], target = 6      -> [0, 1] (same number, different positions: fine)
```

The list has between 2 and 10,000 numbers, which can be negative.

By hand. Walk along the list. At each number, ask "which number would I need to go with this one?" That is `target` minus the current number. Then ask "have I already walked past that number?" If yes, you have your pair. If no, write the current number down (with its position) so later numbers can find it, and move on.

The idea. Keep a dictionary from *number seen* to *its position*. For each position `i`: compute `need = target - nums[i]`; if `need` is in the dictionary, return its stored position and `i`; otherwise store `nums[i] -> i`. Storing *after* checking is what stops a position from pairing with itself.

The Python you would have written.

```
def twoSum(nums, target):
    seen = {}                # value -> index
    for i, x in enumerate(nums):
        need = target - x
        if need in seen:
            return [seen[need], i]
        seen[x] = i
    return []
```

In C++. This is the full file. `main()` at the bottom runs the tests.

```

// LeetCode 1. Two Sum
#include <cassert>
#include <iostream>
#include <unordered_map>
#include <vector>

// One pass: for each x, ask "have I already seen target - x?". If not, remember x.
std::vector<int> twoSum(std::vector<int>& nums, int target) {
    std::unordered_map<int, int> seen; // value -> index
    for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
        int need = target - nums[i];
        auto it = seen.find(need); // find, not [] - [] would insert need with index 0
        if (it != seen.end()) return {it->second, i};
        seen[nums[i]] = i; // insert AFTER checking so we never pair i with itself
    }
    return {}; // LeetCode guarantees a solution; keep the compiler happy
}

int main() {
    std::vector<int> a = {2, 7, 11, 15};
    assert(twoSum(a, 9) == std::vector<int>({0, 1}));

    std::vector<int> b = {3, 2, 4};
    assert(twoSum(b, 6) == std::vector<int>({1, 2}));

    std::vector<int> c = {3, 3}; // duplicate values: must not reuse index
    assert(twoSum(c, 6) == std::vector<int>({0, 1}));

    std::vector<int> d = {-1, -2, -3, -4, -5}; // negatives
    assert(twoSum(d, -8) == std::vector<int>({2, 4}));

    std::vector<int> e = {1}; // no answer -> empty
    assert(twoSum(e, 2).empty());

    std::cout << "OK 01_two_sum.cpp\n";
    return 0;
}

```

What is new:

- `std::vector<int> twoSum(std::vector<int>& nums, int target)` : returns a vector of ints (the two positions) and takes the list by reference (c2) plus an `int`. This is LeetCode's exact signature.
- `std::unordered_map<int, int> seen;` : Python's `seen = {}` with the types stated: keys are ints, values are ints (c6).
- `for (int i = 0; i < static_cast<int>(nums.size()); ++i)` : Python's `for i in range(len(nums))`. The cast is the c1 habit so `int i` and `size()` compare without a warning.
- `auto it = seen.find(need);` and `if (it != seen.end())` : Python's `if need in seen`. `find` hands back an *iterator*, a thing that points at the entry or at `end()` meaning "not there". `it->second` is the value stored (the position). We use `find` and not `seen[need]` because `[]` would insert `need` with position 0 (the c6 trap) and we would "find" a number we never saw.
- `return {it->second, i};` : builds the two-element vector in place. Python's `return [seen[need], i]`.
- `return {};` : an empty vector. Needed because a function that promises to return something must do so on every path, or the compiler warns.

Walk through the example. `nums = [2, 7, 11, 15]`, `target = 9`, `seen` empty. `i = 0`: `need = 9 - 2 = 7`; not in `seen`; store `2 -> 0`. `i = 1`: `need = 9 - 7 = 2`; `find(2)` hits, `it->second` is 0; return `{0, 1}`. Done in two steps.

Complexity. One pass over the list, one dictionary lookup per element: time proportional to the length, $O(n)$. The dictionary can hold every number, so memory is $O(n)$ too.

Robotics. Matching two sensor streams by timestamp is this pattern: index one stream by its stamp in a hash map, then walk the other stream once asking "is there a sample at (or near) the stamp I need?".

Nested loops over both streams would be far too slow at 100 Hz.

Follow-ups you may get. - *The list is sorted; can you use no extra memory?* Two pointers, one at each end, moving inward: if the sum is too small move the left one right, if too big move the right one left. $O(n)$ time, no map. - *Return all pairs, not just one?* Store counts in the map instead of positions, and handle $x ==$ need separately. - *Why `unordered_map` and not `map`?* `map` keeps keys sorted, which costs $O(\log n)$ per lookup; we do not need order, so the hash map's $O(1)$ wins.

2. Contains Duplicate (LeetCode 217, easy)

The problem in plain words. Given a list of whole numbers, return `true` if any number appears two or more times, otherwise `false`.

```
[1, 2, 3, 1] -> 1 appears twice      -> true
[1, 2, 3, 4] -> every number once   -> false
[]           -> nothing can repeat   -> false
[42]         -> one number, once     -> false
```

Up to 100,000 numbers, which can be negative.

By hand. Read the numbers one at a time and keep a list of the ones you have already seen. If the number you are reading is already on your list, stop: duplicate. If you get to the end, no duplicate. Keeping the list as a set makes "is it on my list" instant.

The idea. A set seen. For each number: if it is in seen, return `true`; else add it. After the loop return `false`. In C++ the "is it in, and if not add it" pair is one call: `insert` reports whether the number was new.

The Python you would have written.

```
def containsDuplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

In C++.

```
// LeetCode 217. Contains Duplicate
#include <algorithm>
#include <cassert>
#include <iostream>
#include <unordered_set>
#include <vector>

// Hash set:  $O(n)$  time,  $O(n)$  space. insert() returns {iterator, inserted?}.
bool containsDuplicate(std::vector<int>& nums) {
    std::unordered_set<int> seen;
    seen.reserve(nums.size() * 2); // avoid rehashing; optional but interviewers like it
    for (int x : nums) {
        if (!seen.insert(x).second) return true;
    }
    return false;
}

// Alternative: sort and compare neighbours.  $O(n \log n)$  time,  $O(1)$  extra space.
bool containsDuplicateSort(std::vector<int> nums) { // by value: we may reorder it
    std::sort(nums.begin(), nums.end());
    return std::adjacent_find(nums.begin(), nums.end()) != nums.end();
}
```

```

}

int main() {
    std::vector<int> a = {1, 2, 3, 1};
    assert(containsDuplicate(a) == true);
    assert(containsDuplicateSort(a) == true);

    std::vector<int> b = {1, 2, 3, 4};
    assert(containsDuplicate(b) == false);
    assert(containsDuplicateSort(b) == false);

    std::vector<int> c = {1, 1, 1, 3, 3, 4, 3, 2, 4, 2};
    assert(containsDuplicate(c) == true);

    std::vector<int> d = {}; // empty
    assert(containsDuplicate(d) == false);
    std::vector<int> e = {42}; // single element
    assert(containsDuplicate(e) == false);

    std::cout << "OK 02_contains_duplicate.cpp\n";
    return 0;
}

```

What is new:

- `std::unordered_set<int> seen;` : Python's `seen = set()` for ints (c6).
- `seen.reserve(nums.size() * 2);` : tell the set to make room up front so it does not have to grow several times. Optional; the code is correct without it.
- `if (!seen.insert(x).second) return true;` : `insert` returns a pair whose `.second` is true if `x` was new. `!` flips it: "if it was *not* new, we found a duplicate". This is Python's `if x in seen: return True` and `seen.add(x)` in one line.
- The second function, `containsDuplicateSort`, is the other way to do it: sort the list, then a duplicate must be next to its twin. `std::adjacent_find` walks the sorted list looking for two equal neighbours and returns `end()` if there are none. It takes `nums` *by value* (no `&`, `c2`) on purpose: sorting reorders the list, and the copy keeps the caller's order intact.

Walk through the example. `[1, 2, 3, 1]`: insert 1, new. Insert 2, new. Insert 3, new. Insert 1, `.second` is false, return true. For `[1, 2, 3, 4]` all four inserts are new and the loop ends; return false.

Complexity. Hash set version: one pass, $O(n)$ time and $O(n)$ memory. Sort version: $O(n \log n)$ time, but only the copy for memory.

Robotics. Checking that a list of IDs has no repeats is routine: the landmark IDs a SLAM keyframe observed, the joint names in a robot description file, the node names on a ROS graph. All of them are "insert into a set, watch `.second`".

Follow-ups you may get. - *The values are all between 0 and n ?* Use a `std::vector<bool> seen(n + 1)` indexed by the value: no hashing at all. - *A duplicate within distance k of itself (LeetCode 219)?* Keep a sliding-window set: erase `nums[i - k]` as `i` advances so the set only holds the last k numbers. - *Which version would you choose?* The set for speed and clarity; the sort if memory is tight and reordering is allowed.

3. Valid Anagram (LeetCode 242, easy)

The problem in plain words. Two strings `s` and `t` of lowercase letters. Return true if `t` is a rearrangement of the letters of `s` (same letters, same counts, any order), otherwise false.

<code>s = "anagram"</code>	<code>t = "nagaram"</code>	-> same letters: a a a g m n r	-> true
<code>s = "rat"</code>	<code>t = "car"</code>	-> rat has t, car has c	-> false
<code>s = ""</code>	<code>t = ""</code>	-> nothing versus nothing	-> true
<code>s = "a"</code>	<code>t = "ab"</code>	-> different lengths	-> false

Up to 50,000 letters each, all lowercase a to z.

By hand. Count how many of each letter `s` has, count the same for `t`, and compare the two tallies. On paper you would make a table with 26 rows. A quicker version: go through both strings together, adding one to a letter's row for `s` and taking one away for `t`; if every row ends at zero they match.

The idea. Lengths differ: `false`. Otherwise 26 counters, all zero. For each position `i`, add 1 at `s[i]`'s counter and subtract 1 at `t[i]`'s. Any counter not zero at the end: `false`. The counter for a letter is found with the `'0'` trick from c5: `ch - 'a'` is 0 for `a`, 1 for `b`, and so on.

The Python you would have written.

```
def isAnagram(s, t):
    if len(s) != len(t):
        return False
    count = [0] * 26
    for a, b in zip(s, t):
        count[ord(a) - ord('a')] += 1
        count[ord(b) - ord('a')] -= 1
    return all(c == 0 for c in count)
```

In C++.

```
// LeetCode 242. Valid Anagram
#include <cassert>
#include <iostream>
#include <string>

// Lowercase letters only -> a fixed 26-slot count array beats a hash map.
// +1 for s, -1 for t; anagram iff every slot ends at zero.
bool isAnagram(std::string s, std::string t) {
    if (s.size() != t.size()) return false;
    int count[26] = {}; // {} zero-initialises; plain int count[26]; does not
    for (size_t i = 0; i < s.size(); ++i) {
        ++count[s[i] - 'a'];
        --count[t[i] - 'a'];
    }
    for (int c : count) {
        if (c != 0) return false;
    }
    return true;
}

int main() {
    assert(isAnagram("anagram", "nagaram") == true);
    assert(isAnagram("rat", "car") == false);
    assert(isAnagram("", "") == true); // empty strings
    assert(isAnagram("a", "ab") == false); // length mismatch
    assert(isAnagram("aacc", "ccac") == false); // same letters, different multiset
    std::cout << "OK 03_valid_anagram.cpp\n";
    return 0;
}
```

What is new:

- `int count[26] = {};` : 26 boxes of `int`, all zero. This is a plain C-style array, a vector with a size fixed at compile time. The `= {}` is what zeroes it; `int count[26];` alone leaves garbage inside, unlike Python's `[0] * 26`.
- `for (size_t i = 0; i < s.size(); ++i)` : `size_t` is the type `size()` returns (c1 skip paragraph), so using it for `i` avoids the signedness warning without a cast. Both are fine.
- `++count[s[i] - 'a'];` : `s[i]` is a `char`; `s[i] - 'a'` is its position in the alphabet (c5); `++` adds one to that box. Python's `count[ord(a) - ord('a')] += 1`.
- `for (int c : count)` : range-for works on plain arrays too.

Walk through "rat" and "car". Lengths both 3. $i = 0$: +1 at r, -1 at c. $i = 1$: +1 at a, -1 at a (back to 0). $i = 2$: +1 at t, -1 at r (back to 0). Final non-zero boxes: c is -1, t is +1. Return false. For "anagram" / "nagaram" every box returns to 0 and the answer is true.

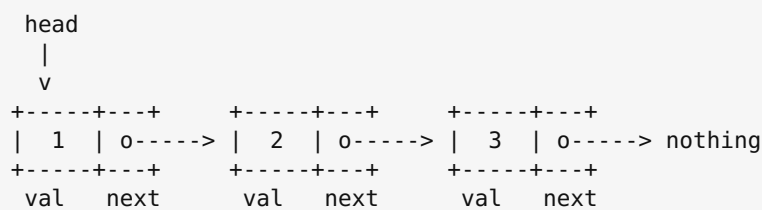
Complexity. One pass over the strings and one over the 26 boxes: $O(n)$ time. Memory is 26 ints no matter how long the strings are: $O(1)$.

Robotics. A fixed-size array indexed by `value - base` is a histogram without a map: counting how many cells of a costmap hold each of 256 cost values, or binning laser intensities. Same `[c - 'a']` arithmetic with a different alphabet.

Follow-ups you may get. - What if the strings can hold any character, not just lowercase? Swap the 26-box array for an `unordered_map<char, int>`; the logic is identical. - Group a list of words into anagram groups (LeetCode 49)? Use the sorted word, or the 26 counts turned into a string, as a map key, and collect words under it. - What breaks if a capital letter sneaks in? 'A' - 'a' is negative, so the index is off the front of the array: undefined behaviour. The constraint promises lowercase; check it if you cannot trust the input.

4. Reverse Linked List (LeetCode 206, easy)

First, what a linked list is. A vector keeps its boxes side by side. A linked list keeps its boxes anywhere in memory, and each box carries an *arrow* to the next box. The whole list is reached by holding the arrow to the first box (the *head*). The last box's arrow points at nothing.



What a pointer is, for now. A pointer is an arrow to a box. Chapter 02 goes deeper; for this chapter you need three things:

- `ListNode*` (with the star) is the type "arrow to a `ListNode` box". `ListNode* head` is a box that holds an arrow, not a node.
- `->` follows the arrow and picks a field. `head->val` is "follow head, read val": 1 in the picture. `head->next` is "follow head, read its arrow": the arrow to the 2 box. Python's `head.val` and `head.next`.
- `nullptr` is "no arrow", the arrow pointing at nothing. Python's `None`. Following it is a crash, so you always check `!= nullptr` first.

The node itself is a `struct`, a box with named fields, which is Python's class without methods:

```

struct ListNode {
    int val;           // the number stored in this box
    ListNode* next;    // the arrow to the next box (nullptr at the end)
};

```

Making boxes and hooking them together:

```

ListNode* a = new ListNode{1, nullptr}; // make a box holding 1; a is the arrow to it
ListNode* b = new ListNode{2, nullptr}; // another box holding 2
a->next = b;                             // the arrow inside a now points at b
assert(a->next->val == 2);                 // follow a, follow its next, read val
assert(b->next == nullptr);              // b points at nothing
delete b;                                // give the boxes back when done (the files do this in deleteList)
delete a;

```

`new` makes a box that outlives the function it was made in, and hands back the arrow to it. `delete` throws the box away. LeetCode does that for you; the test files here do it in a helper.

The problem in plain words. Given the head of a linked list, reverse it so the arrows point the other way, and return the new head (the box that used to be last).

before:	1 -> 2 -> 3 -> 4 -> 5 -> nothing	head is the 1 box
after:	5 -> 4 -> 3 -> 2 -> 1 -> nothing	head is the 5 box
one node:	7 -> nothing	stays 7 -> nothing
empty list:	nothing	stays nothing

Up to 5,000 nodes.

By hand. Walk along the list one box at a time. At each box, turn its arrow around so it points at the box you just came from. The catch: the moment you turn the arrow, you lose the way forward. So before turning it, write down where it pointed. Three pieces of paper: *prev* (the box behind me), *cur* (the box I am on), *next* (where I was about to go).

start:	prev = nothing	cur = 1	1 -> 2 -> 3
step 1:	next = 2	turn 1's arrow: 1 -> nothing	prev = 1, cur = 2
step 2:	next = 3	turn 2's arrow: 2 -> 1	prev = 2, cur = 3
step 3:	next = nothing	turn 3's arrow: 3 -> 2	prev = 3, cur = nothing
cur is nothing: stop. prev (the 3 box) is the new head. 3 -> 2 -> 1 -> nothing			

The idea. `prev = null`, `cur = head`. While `cur` is not null: save `next = cur.next`; point `cur.next` at `prev`; move both forward (`prev = cur`, `cur = next`). Return `prev`.

The Python you would have written.

```
def reverseList(head):
    prev = None
    cur = head
    while cur is not None:
        nxt = cur.next          # save before we overwrite
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

In C++.

```
// LeetCode 206. Reverse Linked List
#include <cassert>
#include <iostream>
#include <vector>

// LeetCode's definition, verbatim.
struct ListNode {
    int val;
    ListNode* next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode* next) : val(x), next(next) {}
};

// Iterative: walk the list, flipping each `next` to point backwards.
// Three pointers: prev (already reversed), cur (being flipped), next (saved before we overwrite).
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    ListNode* cur = head;
    while (cur != nullptr) {
        ListNode* next = cur->next;    // save before we lose it
        cur->next = prev;
        prev = cur;
        cur = next;
    }
```

```

    }
    return prev;                                // prev is the new head (nullptr for an empty list)
}

// Recursive: reverse everything after head, then hook head onto the tail.
// O(n) stack depth – mention that to the interviewer.
ListNode* reverseListRecursive(ListNode* head) {
    if (head == nullptr || head->next == nullptr) return head;
    ListNode* newHead = reverseListRecursive(head->next);
    head->next->next = head;                        // the node after us now points back at us
    head->next = nullptr;                         // we become the tail (until a caller re-hooks us)
    return newHead;
}

// ---- test helpers -----
ListNode* buildList(const std::vector<int>& vals) {
    ListNode dummy;
    ListNode* tail = &dummy;
    for (int v : vals) {
        tail->next = new ListNode(v);
        tail = tail->next;
    }
    return dummy.next;
}

std::vector<int> toVector(ListNode* head) {
    std::vector<int> out;
    for (; head != nullptr; head = head->next) out.push_back(head->val);
    return out;
}

void deleteList(ListNode* head) {
    while (head != nullptr) {
        ListNode* next = head->next;
        delete head;
        head = next;
    }
}

int main() {
    {
        ListNode* h = buildList({1, 2, 3, 4, 5});
        h = reverseList(h);
        assert(toVector(h) == std::vector<int>({5, 4, 3, 2, 1}));
        h = reverseListRecursive(h);                // reverse back
        assert(toVector(h) == std::vector<int>({1, 2, 3, 4, 5}));
        deleteList(h);
    }
    {
        ListNode* h = buildList({1, 2});
        h = reverseListRecursive(h);
        assert(toVector(h) == std::vector<int>({2, 1}));
        deleteList(h);
    }
    {
        ListNode* h = buildList({7});                // single node
        assert(toVector(reverseList(h)) == std::vector<int>({7}));
        deleteList(h);
    }
    assert(reverseList(nullptr) == nullptr);          // empty list
    assert(reverseListRecursive(nullptr) == nullptr);

    std::cout << "OK 04_reverse_linked_list.cpp\n";
    return 0;
}

```

What is new:

- The struct `ListNode` at the top is LeetCode's, with three *constructors* (the lines starting `ListNode(...)`): ways to make a node with no value, with a value, or with a value and a next arrow. : `val(x)`, `next(nullptr)` sets the fields. You will always be given this struct; you do not write it.
- `ListNode* reverseList(ListNode* head)` : takes an arrow, returns an arrow.
- `ListNode* prev = nullptr;` : Python's `prev = None`.
- `while (cur != nullptr)` : Python's `while cur is not None`.
- `ListNode* next = cur->next;` : Python's `nxt = cur.next`. -> follows the arrow.
- `reverseListRecursive` is the second way: reverse everything after head, then hook head on at the end. `head->next->next = head` reads "the box after me now points back at me". It uses one stack frame per node, so for 5,000 nodes it is fine and for a million it is not.
- `buildList`, `toVector`, `deleteList` are helpers for the tests only: make a list from a vector, read a list into a vector, throw the boxes away.

Walk through 1 -> 2 -> 3. `prev = null, cur = 1`. Round 1: `next = 2; 1.next = null; prev = 1; cur = 2`. Round 2: `next = 3; 2.next = 1; prev = 2; cur = 3`. Round 3: `next = null; 3.next = 2; prev = 3; cur = null`. Loop ends. Return `prev`, the 3 box: `3 -> 2 -> 1`.

Complexity. Visits each box once: $O(n)$ time. The loop version uses three arrows no matter how long the list is: $O(1)$ memory. The recursive version uses $O(n)$ stack.

Robotics. Following `next` arrows box by box is how you walk any linked structure: climbing a transform tree from a wheel frame up through `parent` links to the map frame, or walking a chain of message buffers in a zero-copy transport. The discipline "save the arrow before you overwrite it" is the entire skill.

Follow-ups you may get. - *What if you forget to save `next`?* After `cur->next = prev` the rest of the list is unreachable; you reverse one box and lose the rest. - *Reverse only positions m to n (LeetCode 92)?* Walk to box $m-1$, run this same loop for $n-m+1$ boxes, then reconnect the two ends. - *Why does the caller have to write `h = reverseList(h)`?* After reversing, the old head is the tail. If the caller keeps using its old arrow it sees a one-box list.

5. Merge Two Sorted Lists (LeetCode 21, easy)

The problem in plain words. Two linked lists, each already sorted from small to large. Join them into one sorted list by re-pointing the existing boxes (do not make new ones) and return the head of the result.

```
list1:  1 -> 2 -> 4
list2:  1 -> 3 -> 4
result: 1 -> 1 -> 2 -> 3 -> 4 -> 4

list1 empty, list2 = 0      -> 0
both empty                -> empty
list1 = 5 -> 6 -> 7, list2 = 1 -> 2      -> 1 -> 2 -> 5 -> 6 -> 7
```

Each list has up to 50 nodes.

By hand. Put a finger on the front of each list. Compare the two boxes under your fingers, take the smaller one, attach it to the end of the result, and move that finger forward. Repeat until one list runs out. Then attach the whole rest of the other list in one go (it is already sorted).

The awkward part is "attach to the end of the result" when the result is still empty: there is no end yet. The trick is a *dummy* box: a fake first box that holds nothing useful. You always attach after it, and at the end the real answer is whatever comes after the dummy.

```
dummy -> nothing           tail = dummy
compare 1 (list1) and 1 (list2): take list1's 1  dummy -> 1           tail = that 1
compare 2 and 1: take list2's 1                  dummy -> 1 -> 1           tail = second 1
compare 2 and 3: take 2                          dummy -> 1 -> 1 -> 2
```

```

compare 4 and 3: take 3                dummy -> 1 -> 1 -> 2 -> 3
compare 4 and 4: take list1's 4 (ties go to list1) dummy -> 1 -> 1 -> 2 -> 3 -> 4
list1 is empty: attach the rest of list2 (4)    dummy -> 1 -> 1 -> 2 -> 3 -> 4 -> 4
answer: whatever dummy points at

```

The idea. `dummy` node, `tail = dummy`. While both lists are non-empty: hang the smaller head on `tail.next`, advance that list, advance `tail`. Then `tail.next = whichever list is left`. Return `dummy.next`.

The Python you would have written.

```

def mergeTwoLists(l1, l2):
    dummy = ListNode()           # a fake first node so there is always a tail
    tail = dummy
    while l1 is not None and l2 is not None:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 if l1 is not None else l2
    return dummy.next

```

In C++.

```

// LeetCode 21. Merge Two Sorted Lists
#include <cassert>
#include <iostream>
#include <vector>

struct ListNode {
    int val;
    ListNode* next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode* next) : val(x), next(next) {}
};

// Dummy-head idiom: a stack-allocated sentinel node means "append to tail"
// never has to special-case the empty result. Return dummy.next.
ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
    ListNode dummy;           // lives on the stack, never returned itself
    ListNode* tail = &dummy;
    while (list1 != nullptr && list2 != nullptr) {
        if (list1->val <= list2->val) { // <= keeps the merge stable
            tail->next = list1;
            list1 = list1->next;
        } else {
            tail->next = list2;
            list2 = list2->next;
        }
        tail = tail->next;
    }
    tail->next = (list1 != nullptr) ? list1 : list2; // splice the leftover run
    return dummy.next;
}

// ---- test helpers -----
ListNode* buildList(const std::vector<int>& vals) {
    ListNode dummy;
    ListNode* tail = &dummy;
    for (int v : vals) {
        tail->next = new ListNode(v);
        tail = tail->next;
    }
    return dummy.next;
}

```

```

}
std::vector<int> toVector(ListNode* head) {
    std::vector<int> out;
    for (; head != nullptr; head = head->next) out.push_back(head->val);
    return out;
}
void deleteList(ListNode* head) {
    while (head != nullptr) {
        ListNode* next = head->next;
        delete head;
        head = next;
    }
}

int main() {
    {
        ListNode* m = mergeTwoLists(buildList({1, 2, 4}), buildList({1, 3, 4}));
        assert(toVector(m) == std::vector<int>({1, 1, 2, 3, 4, 4}));
        deleteList(m);
    }
    assert(mergeTwoLists(nullptr, nullptr) == nullptr);           // both empty
    {
        ListNode* m = mergeTwoLists(nullptr, buildList({0}));    // one empty
        assert(toVector(m) == std::vector<int>({0}));
        deleteList(m);
    }
    {
        ListNode* m = mergeTwoLists(buildList({5, 6, 7}), buildList({1, 2})); // no interleaving
        assert(toVector(m) == std::vector<int>({1, 2, 5, 6, 7}));
        deleteList(m);
    }
    std::cout << "OK 05_merge_two_sorted_lists.cpp\n";
    return 0;
}

```

What is new:

- `ListNode dummy;` : a node made *without new*. It is an ordinary local box (no star in the type) that disappears when the function ends. That is fine because we never return the dummy itself, only what comes after it.
- `ListNode* tail = &dummy;` : `&` in front of a variable means "the arrow to this box". So `tail` is an arrow pointing at `dummy`. (This `&` is a different use from the `&` in a parameter type, `c2`; both mean "refer to the real thing rather than copy it".)
- `list1->val <= list2->val` : follow each arrow, compare the numbers. `<=` rather than `<` means ties take from `list1`, which keeps equal elements in their original order.
- `tail->next = (list1 != nullptr) ? list1 : list2;` : the one-line if from `c3`. "If `list1` still has boxes, attach `list1`, else attach `list2`."
- `return dummy.next;` : `.` not `->`, because `dummy` is a box, not an arrow to a box. Python's `return dummy.next`.

Walk through the example. Follow the hand trace above; the code does exactly those steps. After the loop `list1` is `nullptr` and `list2` is the last 4, so the ternary attaches `list2`. `dummy.next` is the first 1, which is returned.

Complexity. Each box is looked at once: $O(n + m)$ time for lists of length n and m . No new boxes are made: $O(1)$ extra memory.

Robotics. Merging two time-sorted streams is message synchronisation: combining a lidar scan stream and an odometry stream into one timeline ordered by stamp is this loop with `stamp` in place of `val`. The dummy-head trick reappears any time you build a linked result box by box.

Follow-ups you may get. - *Merge k sorted lists (LeetCode 23)?* Keep the current head of every list in a min-heap keyed by value; pop the smallest, push its successor. $O(N \log k)$. - *Why not return &dummy?* dummy is destroyed when the function returns; the arrow would point at a dead box. - *Recursive version?* The smaller head's next is `merge(rest of that list, other list)`. Same time, $O(n + m)$ stack.

6. Maximum Subarray (LeetCode 53, medium)

The problem in plain words. Given a list of whole numbers (some negative), find the *contiguous* run of one or more neighbouring numbers whose sum is largest, and return that sum. You do not have to return the run itself.

```
nums = [ -2,  1, -3,  4, -1,  2,  1, -5,  4 ]
           ^^^^^^^^^^^^^^^^^
           4 + (-1) + 2 + 1 = 6      -> answer 6

[1]                -> 1          (the only run)
[5, 4, -1, 7, 8] -> 23          (the whole list)
[-3, -1, -2]      -> -1          (all negative: the best run is the single number -1)
```

Up to 100,000 numbers, each between -10,000 and 10,000.

By hand. Walk left to right, keeping a running sum of "the best run that ends right here". At each number you have two choices: extend the run you were carrying (add this number to it) or throw that run away and start a new one at this number. Starting fresh is better exactly when the run you were carrying had gone negative, because a negative prefix only drags you down. Keep the best running sum you ever saw.

number:	-2	1	-3	4	-1	2	1	-5	4	
run here:	-2	1	-2	4	3	5	6	1	5	
best so far:	-2	1	1	4	4	5	6	6	6	-> 6

"run here" for 1 is $\max(1, -2 + 1) = 1$: start fresh. For 4 it is $\max(4, -2 + 4) = 4$: fresh again. For -1 it is $\max(-1, 4 + -1) = 3$: extend.

The idea. `cur = best = nums[0]`. For each later number `x`: `cur = max(x, cur + x)`; `best = max(best, cur)`. Return `best`. Starting `best` at `nums[0]`, not at 0, is what makes the all-negative case right. This is known as Kadane's algorithm.

The Python you would have written.

```
def maxSubArray(nums):
    best = nums[0]
    cur = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)      # extend the run, or start again at x
        best = max(best, cur)
    return best
```

In C++.

```
// LeetCode 53. Maximum Subarray (Kadane)
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// Kadane: best sum ending at i is either "extend the previous run" or "start fresh at i".
// Track the best of those over the whole array.
int maxSubArray(std::vector<int>& nums) {
    int best = nums[0];           // NOT 0 – an all-negative array must return its largest element
    int cur = nums[0];
    for (size_t i = 1; i < nums.size(); ++i) {
```

```

        cur = std::max(nums[i], cur + nums[i]);
        best = std::max(best, cur);
    }
    return best;
}

int main() {
    std::vector<int> a = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
    assert(maxSubArray(a) == 6); // [4,-1,2,1]

    std::vector<int> b = {1};
    assert(maxSubArray(b) == 1); // single element

    std::vector<int> c = {5, 4, -1, 7, 8};
    assert(maxSubArray(c) == 23); // whole array

    std::vector<int> d = {-3, -1, -2};
    assert(maxSubArray(d) == -1); // all negative: initialising best=0 would give 0

    std::cout << "OK 06_maximum_subarray.cpp\n";
    return 0;
}

```

What is new:

- `#include <algorithm>` gives `std::max` and `std::min`.
- `for (size_t i = 1; i < nums.size(); ++i)` : Python's `for x in nums[1:]` written with an index, starting from 1.
- `cur = std::max(nums[i], cur + nums[i]);` : Python's `max(x, cur + x)`. Both arguments must have the same type (int and int here).

Walk through the example. See the table above. `cur` and `best` start at -2. Then `x = 1`: `cur = max(1, -1) = 1`, `best = 1`. `x = -3`: `cur = max(-3, -2) = -2`, `best` stays 1. `x = 4`: `cur = 4`, `best = 4`. `x = -1`: `cur = 3`. `x = 2`: `cur = 5`, `best = 5`. `x = 1`: `cur = 6`, `best = 6`. `x = -5`: `cur = 1`. `x = 4`: `cur = 5`. Return 6.

Complexity. One pass, two variables: $O(n)$ time, $O(1)$ memory.

Robotics. "The strongest contiguous stretch" of a 1D signal: the longest run of a laser scan above a reflectance threshold, or the window of accelerometer samples with the most energy when detecting a bump. Same running-sum trick.

Follow-ups you may get. - *Why not start `best` at 0?* Then `[-3, -1, -2]` would return 0, but an empty run is not allowed; the answer must be -1. - *Return the start and end positions too?* Remember where the current run started (reset it whenever you start fresh) and copy those positions whenever `best` improves. - *Could the sum overflow an `int`?* 100,000 numbers of at most 10,000 is at most one billion, which fits. With bigger limits, use `long long`.

7. Move Zeroes (LeetCode 283, easy)

The problem in plain words. Given a list of whole numbers, move every 0 to the end of the list, keeping the other numbers in their original order. Do it *in place*: change the given list, do not build a new one, and return nothing.

```

[0, 1, 0, 3, 12]  ->  [1, 3, 12, 0, 0]
[0]               ->  [0]
[1, 2, 3]         ->  [1, 2, 3]      (nothing to do)
[0, 0, 1]         ->  [1, 0, 0]

```

Up to 10,000 numbers.

By hand. Keep a marker for "the next free slot at the front", starting at position 0. Walk through the list; each time you meet a non-zero, swap it into the free slot and move the marker one to the right. Zeros are never picked up, so they naturally end up in the slots after the marker.

read ->	0	1	0	3	12		write = 0
read=0:	0 is zero, skip						write = 0
read=1:	1 non-zero, swap slots 0,1					[1, 0, 0, 3, 12]	write = 1
read=2:	0 is zero, skip						write = 1
read=3:	3 non-zero, swap slots 1,3					[1, 3, 0, 0, 12]	write = 2
read=4:	12 non-zero, swap slots 2,4					[1, 3, 12, 0, 0]	write = 3

The idea. Two indices: `read` scans every position, `write` is the next slot for a non-zero. When `nums[read]` is non-zero, swap `nums[write]` and `nums[read]`, then `write += 1`.

The Python you would have written.

```
def moveZeroes(nums):
    write = 0
    for read in range(len(nums)):
        if nums[read] != 0:
            nums[write], nums[read] = nums[read], nums[write]
            write += 1
```

In C++.

```
// LeetCode 283. Move Zeroes
#include <cassert>
#include <iostream>
#include <utility>
#include <vector>

// Two pointers: `write` is the slot for the next non-zero. Swapping (instead of
// overwriting then zero-filling) keeps it a single pass and preserves order.
void moveZeroes(std::vector<int>& nums) {
    int write = 0;
    for (int read = 0; read < static_cast<int>(nums.size()); ++read) {
        if (nums[read] != 0) {
            std::swap(nums[write], nums[read]); // no-op when write == read
            ++write;
        }
    }
}

int main() {
    std::vector<int> a = {0, 1, 0, 3, 12};
    moveZeroes(a);
    assert(a == std::vector<int>({1, 3, 12, 0, 0}));

    std::vector<int> b = {0};
    moveZeroes(b);
    assert(b == std::vector<int>({0}));

    std::vector<int> c = {}; // empty
    moveZeroes(c);
    assert(c.empty());

    std::vector<int> d = {1, 2, 3}; // no zeroes: order untouched
    moveZeroes(d);
    assert(d == std::vector<int>({1, 2, 3}));

    std::vector<int> e = {0, 0, 1};
    moveZeroes(e);
    assert(e == std::vector<int>({1, 0, 0}));

    std::cout << "OK 07_move_zeroes.cpp\n";
```

```
    return 0;
}
```

What is new:

- `void moveZeroes(std::vector<int>& nums)` : returns nothing (void) and takes the list *by reference*. The `&` is the whole problem: without it the function would sort a copy and the caller's list would not change (c2).
- `std::swap(nums[write], nums[read]);` : Python's `a, b = b, a`. Needs `#include <utility>`. When `write == read` it swaps a box with itself, which is harmless.

Walk through the example. See the table above; the code does exactly those swaps. The function ends with `nums` equal to `[1, 3, 12, 0, 0]` and returns nothing.

Complexity. One pass, one swap per non-zero: $O(n)$ time, $O(1)$ memory.

Robotics. Compacting a buffer in place is how you drop bad points from a point cloud (NaN ranges, zero-intensity returns) without allocating a second cloud: the same read/write sweep, usually spelled `std::remove_if` followed by `erase`.

Follow-ups you may get. - *Fewer writes?* Copy each non-zero forward to `nums[write]` (no swap), then fill from `write` to the end with zeros. Fewer stores when zeros are rare. - *A one-liner from the library?* `std::stable_partition(nums.begin(), nums.end(), [](int x) { return x != 0; });` keeps order and moves the zeros to the back. - *What if the function took `std::vector<int> nums` without `&`?* It compiles, runs, and the caller's vector is unchanged. The tests in `main` would fail.

8. Best Time to Buy and Sell Stock (LeetCode 121, easy)

The problem in plain words. A list of prices, one per day. You may buy on one day and sell on a *later* day, once. Return the biggest profit you can make. If no profit is possible, return 0 (you simply do not trade).

```
prices = [7, 1, 5, 3, 6, 4]
day      0  1  2  3  4  5

buy on day 1 at 1, sell on day 4 at 6  -> profit 5      -> answer 5
(buying at 1 and selling at 7 is not allowed: day 0 is before day 1)

[7, 6, 4, 3, 1]  -> prices only fall  -> 0
[5]              -> one day, cannot sell after buying  -> 0
[2, 4, 1]        -> buy 2, sell 4      -> 2 (the later 1 is cheaper but nothing comes after it)
```

Up to 100,000 days.

By hand. Walk through the days. Keep two notes: the cheapest price you have seen so far, and the best profit so far. On each day, "sell today after buying at the cheapest price so far" is today's price minus that minimum; if that beats the best, write it down. Then, if today is cheaper than the minimum, update the minimum.

day:	0	1	2	3	4	5	
price:	7	1	5	3	6	4	
min so far:	7	1	1	1	1	1	
best:	0	0	4	4	5	5	-> 5

The idea. `minPrice = prices[0]`, `best = 0`. For each later price `p`: `best = max(best, p - minPrice)` and then `minPrice = min(minPrice, p)`. The order matters: update `best` first, so you never "sell" on the same day you set the minimum.

The Python you would have written.

```
def maxProfit(prices):
    min_price = prices[0]
    best = 0
```

```

for p in prices[1:]:
    best = max(best, p - min_price)
    min_price = min(min_price, p)
return best

```

In C++.

```

// LeetCode 121. Best Time to Buy and Sell Stock
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// One pass: the best sale at day i uses the cheapest buy seen so far.
// Track minPrice and the best profit; never sell before you buy.
int maxProfit(std::vector<int>& prices) {
    int minPrice = prices[0];
    int best = 0; // "no transaction" is always allowed
    for (size_t i = 1; i < prices.size(); ++i) {
        best = std::max(best, prices[i] - minPrice);
        minPrice = std::min(minPrice, prices[i]);
    }
    return best;
}

int main() {
    std::vector<int> a = {7, 1, 5, 3, 6, 4};
    assert(maxProfit(a) == 5); // buy 1, sell 6

    std::vector<int> b = {7, 6, 4, 3, 1};
    assert(maxProfit(b) == 0); // monotone decreasing: never buy

    std::vector<int> c = {5};
    assert(maxProfit(c) == 0); // single day

    std::vector<int> d = {2, 4, 1};
    assert(maxProfit(d) == 2); // the later min (1) must not be paired with earlier 4

    std::cout << "OK 08_best_time_to_buy_and_sell_stock.cpp\n";
    return 0;
}

```

What is new: nothing beyond problem 6. `std::max` and `std::min` from `<algorithm>`, an index loop from 1, and `prices[i] - minPrice` as the sell-today profit.

Walk through the example. `minPrice = 7`, `best = 0`. Day 1, price 1: profit `1 - 7 = -6`, `best` stays 0; `min` becomes 1. Day 2, price 5: profit 4, `best = 4`. Day 3, price 3: profit 2, no change. Day 4, price 6: profit 5, `best = 5`. Day 5, price 4: profit 3, no change. Return 5.

Complexity. One pass, two variables: $O(n)$ time, $O(1)$ memory.

Robotics. "Current value minus the running minimum" computed online is the largest rise since the lowest point so far: the worst altitude gain after a dip in a barometer stream, or the largest drift excursion in an odometry error signal, without storing the history.

Follow-ups you may get. - Allowed to buy and sell many times (LeetCode 122)? Add up every positive day-to-day difference. - Return the buy day and sell day? Remember the index of `minPrice`, and copy it (plus today's index) whenever `best` improves. - Why not two nested loops trying every pair? Correct but $O(n^2)$; 100,000 days is ten billion pairs.

9. Valid Parentheses (LeetCode 20, easy)

The problem in plain words. A string made only of the six characters `(, ), [, ], {, }`. Return `true` if every opening bracket is closed by the matching kind of bracket, in the right order (the most recently opened bracket must be the first one closed). Otherwise `false`.

```
"()"      -> true
"()[{}]"  -> true
"([])"    -> true      [ ] is closed inside ( )
"{}"      -> false      { is closed by }
"([)]"    -> false      [ is still open when ) arrives
"{"       -> false      never closed
""        -> true       nothing to check
```

Up to 10,000 characters.

By hand. Read left to right and keep a pile of "brackets still open". An opener goes on top of the pile. A closer must match the bracket on top of the pile; if it does, take that opener off the pile; if it does not (or the pile is empty), the string is bad. At the end the pile must be empty.

```
"([])":  ( -> pile: (
         [ -> pile: ( [
         ] -> top is [, matches, remove -> pile: (
         ) -> top is (, matches, remove -> pile: empty -> true

"{}":    ( -> pile: (
         ] -> top is (, does not match ] -> false
```

The idea. A stack is a pile of plates: you put a plate on top and you take from the top, so the last one on is the first one off. For each character: if it is a closer, check the top of the stack is the matching opener and pop it, else return `false`; if it is an opener, push it. Return whether the stack is empty at the end.

The Python you would have written.

```
def isValid(s):
    close_to_open = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in s:
        if ch in close_to_open:           # a closer
            if not stack or stack[-1] != close_to_open[ch]:
                return False
            stack.pop()
        else:                             # an opener
            stack.append(ch)
    return not stack
```

In C++.

```
// LeetCode 20. Valid Parentheses
#include <cassert>
#include <iostream>
#include <stack>
#include <string>
#include <unordered_map>

// Stack of pending openers. A closer must match the most recent opener.
bool isValid(std::string s) {
    static const std::unordered_map<char, char> closeToOpen = {
        {')', '('}, {']', '['}, {'}', '{'}};
    std::stack<char> st;
    for (char c : s) {
        auto it = closeToOpen.find(c);
        if (it == closeToOpen.end()) {
            st.push(c);           // opener
        } else {
            st.pop();
        }
    }
    return st.empty();
}
```

```

        if (st.empty() || st.top() != it->second) return false;
        st.pop(); // std::stack::pop() returns void – read top() first
    }
}
return st.empty(); // leftover openers mean unbalanced
}

int main() {
    assert(isValid("(") == true);
    assert(isValid("()[{}]" == true);
    assert(isValid("]") == false);
    assert(isValid("([])" == true);
    assert(isValid("") == true); // empty string is valid
    assert(isValid("(") == false); // unmatched opener
    assert(isValid(")") == false); // closer with empty stack
    assert(isValid("([])" == false); // interleaved
    std::cout << "OK 09_valid_parentheses.cpp\n";
    return 0;
}

```

What is new:

- `static const std::unordered_map<char, char> closeToOpen = {...};` : the same dictionary as Python, from closing character to opening character. `const` because it never changes; `static` so it is built once, the first time the function runs, not on every call.
- `std::stack<char> st;` : a stack of characters, from `#include <stack>`. `push` puts one on top, `top()` looks at the top, `pop()` removes the top. `pop()` returns nothing, so you always `top()` first if you need the value. Python's list with `append` and `pop()`.
- `auto it = closeToOpen.find(c); if (it == closeToOpen.end())` : Python's `if ch in close_to_open` turned around: `end()` means "not a closer, so an opener".
- `if (st.empty() || st.top() != it->second) return false;` : the two ways a closer can fail: nothing to close, or the wrong thing on top. `it->second` is the matching opener.
- `return st.empty();` : Python's `return not stack`.

Walk through "([])". (: push, stack (. [: push, stack ([.) : it is a closer whose opener is (; top is [; mismatch; return false.

Complexity. One pass, each character pushed and popped at most once: $O(n)$ time, $O(n)$ memory for the stack.

Robotics. A stack of pending openers is a tiny parser. Checking that `<link>` and `<joint>` tags nest properly in a URDF, or that a behaviour-tree XML has matching begin and end markers, is this loop with tag names instead of brackets.

Follow-ups you may get. - *Only one kind of bracket?* A counter is enough: add one for (, subtract for), fail if it ever goes negative, succeed if it ends at 0. - *Minimum insertions to make it valid?* Same stack; count closers that find nothing to match, then add the openers left on the stack. - *What happens if you call `st.top()` on an empty stack?* Undefined behaviour, usually a crash. That is why `st.empty()` is checked first.

10. Binary Search (LeetCode 704, easy)

The problem in plain words. A list of whole numbers sorted from small to large, and a `target`. Return the position of `target` in the list, or -1 if it is not there. Do it in $O(\log n)$ time, meaning: halve the amount you still have to look at with every step.

```

nums   = [ -1,  0,  3,  5,  9, 12 ]    target = 9    ->  4
index   0   1   2   3   4   5

same list, target = 2                    -> -1 (not present)

```

```
target = -1 -> 0      target = 12 -> 5      (the ends)
[]          -> -1     [5], target 5 -> 0
```

Up to 10,000 numbers, all different.

By hand. Think of finding a word in a dictionary. Open in the middle. If the word you want comes later, ignore the whole first half; if earlier, ignore the whole second half. Open the middle of what is left. Each look throws away half of what remains, so 10,000 entries take about 14 looks.

```
target 9:
lo=0 hi=5 mid=2 nums[2]=3 < 9 -> 9 is to the right: lo = 3
lo=3 hi=5 mid=4 nums[4]=9 == 9 -> found at 4

target 2:
lo=0 hi=5 mid=2 nums[2]=3 > 2 -> 2 is to the left: hi = 1
lo=0 hi=1 mid=0 nums[0]=-1 < 2 -> right: lo = 1
lo=1 hi=1 mid=1 nums[1]=0 < 2 -> right: lo = 2
lo=2 hi=1 lo > hi: nothing left -> -1
```

The idea. `lo = 0`, `hi = n - 1`, the range still possible (both ends included). While `lo <= hi`: `mid` is the middle; if `nums[mid]` is the target return `mid`; if smaller, the target is right of `mid` so `lo = mid + 1`; if larger, `hi = mid - 1`. If the loop ends, `-1`.

The Python you would have written.

```
def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

In C++.

```
// LeetCode 704. Binary Search
#include <algorithm>
#include <cassert>
#include <iostream>
#include <vector>

// Classic closed interval [lo, hi]. Loop while lo <= hi.
int search(std::vector<int>& nums, int target) {
    int lo = 0;
    int hi = static_cast<int>(nums.size()) - 1; // cast first: size() - 1 on empty wraps
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // (lo + hi) / 2 can overflow int
        if (nums[mid] == target) return mid;
        if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

// The lower_bound idiom: first index whose value is >= target, in [0, n].
// Half-open [lo, hi); loop while lo < hi. This is what you actually reuse in
// "first true" / "insert position" / "count of elements < x" questions.
int lowerBound(const std::vector<int>& nums, int target) {
    int lo = 0;
    int hi = static_cast<int>(nums.size());
    while (lo < hi) {
```

```

    int mid = lo + (hi - lo) / 2;
    if (nums[mid] < target) lo = mid + 1;
    else hi = mid;
}
return lo;
}

// upper_bound: first index whose value is > target. Only the comparison changes.
int upperBound(const std::vector<int>& nums, int target) {
    int lo = 0;
    int hi = static_cast<int>(nums.size());
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] <= target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

int main() {
    std::vector<int> a = {-1, 0, 3, 5, 9, 12};
    assert(search(a, 9) == 4);
    assert(search(a, 2) == -1);
    assert(search(a, -1) == 0);           // first element
    assert(search(a, 12) == 5);          // last element

    std::vector<int> empty;
    assert(search(empty, 1) == -1);       // empty: hi = -1, loop never runs
    std::vector<int> one = {5};
    assert(search(one, 5) == 0);
    assert(search(one, 4) == -1);

    // lower/upper bound vs the STL versions
    std::vector<int> b = {1, 2, 2, 2, 5, 7};
    assert(lowerBound(b, 2) == 1);
    assert(upperBound(b, 2) == 4);
    assert(upperBound(b, 2) - lowerBound(b, 2) == 3); // count of 2s
    assert(lowerBound(b, 3) == 4);                    // insert position for 3
    assert(lowerBound(b, 100) == 6);                   // == size when nothing is >= target
    assert(lowerBound(b, 0) == 0);
    assert(std::lower_bound(b.begin(), b.end(), 2) - b.begin() == 1);
    assert(std::upper_bound(b.begin(), b.end(), 2) - b.begin() == 4);
    assert(std::binary_search(b.begin(), b.end(), 5) == true);

    std::cout << "OK 10_binary_search.cpp\n";
    return 0;
}

```

What is new:

- `int hi = static_cast<int>(nums.size()) - 1;` : cast *before* subtracting. On an empty list `nums.size() - 1` would roll over to a huge number (c1 skip paragraph); casting first gives -1, so `lo <= hi` is false and the loop never runs.
- `int mid = lo + (hi - lo) / 2;` : Python's `(lo + hi) // 2`, rearranged so `lo + hi` is never computed. With both near 2 billion, `lo + hi` would overflow an `int`; `hi - lo` cannot.
- `lowerBound` and `upperBound` are a second shape of the same search that you will reuse constantly: instead of "where is target", they answer "where is the first number that is `>= target`" (or `> target`). They use a half-open range, `hi = n` and `while (lo < hi)`, and return `n` if no such number exists. The `main` checks them against the library's `std::lower_bound` and `std::upper_bound`, which do the same thing on any sorted vector.

Walk through target 9. `lo = 0, hi = 5. mid = 0 + 5 / 2 = 2, nums[2] = 3 < 9, lo = 3. mid = 3 + 2 / 2 = 4, nums[4] = 9, return 4.` Two looks for six numbers.

Complexity. The range halves each step: $O(\log n)$ time. Three ints: $O(1)$ memory.

Robotics. `lower_bound` on a sorted vector of timestamps finds the pose recorded just before a lidar scan's stamp, so you can interpolate. A trajectory follower uses the same call to find which segment of a time-stamped path the current time falls in. You will type this loop from memory many times.

Follow-ups you may get. - *First and last position of a value that may repeat (LeetCode 34)?*

`lowerBound(target)` and `upperBound(target) - 1`. - *Why not $(lo + hi) / 2$? It overflows when `lo + hi` passes 2,147,483,647. $lo + (hi - lo) / 2$ gives the same `mid` safely.* - *What goes wrong if you mix the two shapes? `while (lo <= hi)` with `hi = n` reads past the end; `while (lo < hi)` with `hi = mid - 1` can skip the answer. Pick one shape and stick to it.*

Part C — Check yourself

Answer each one in plain words, without looking, before you mark this chapter done.

1. What are the two steps to get a C++ program running, and what does `g++` produce?
2. What is `-7 / 2` in C++, and what is `-7 // 2` in Python? Why are they different?
3. `int a = 100000, b = 100000; long long p = a * b;` gives the wrong answer. Why, and what is the one-word fix in the middle of that line?
4. Draw the two boxes for `void f(int x)` and `void f(int& x)`. Which one lets `f` change the caller's variable?
5. When would you write `const std::vector<int>&` as a parameter instead of `std::vector<int>?`
6. `for (auto row : grid) row[0] = 1;` leaves `grid` unchanged. Why? What is the fix?
7. Write the line that makes a grid of 3 rows and 4 columns, all zero, and explain each part of it.
8. What is the difference between `std::vector<int> v(5, 0)` and `std::vector<int> v{5, 0}?`
9. What does `s.substr(2, 3)` give for `s = "hello world"`, and what are its two arguments?
10. What does `s.find('z')` return when there is no `z`, and why is `if (s.find('z') >= 0)` always true?
11. Why is `'7' - '0'` equal to 7? What number is `'a'`?
12. `ages["bob"]` when there is no `"bob"` in the map: what happens? How do you check for a key without that side effect?
13. In a linked list, what do `ListNode*`, `->` and `nullptr` mean? What does `head->next->val` read?
14. In Reverse Linked List, why must you save `cur->next` before writing `cur->next = prev?`
15. Write `mid` for binary search so it cannot overflow, and say what `static_cast<int>(nums.size()) - 1` protects against.